

---

# **XnslTree**

**Version 1.2.1**

## **Reference Manual**

**Nova Software Labs**

---

© Copyright Nova Software Labs S.A.R.L. 1993 -2002.

All rights reserved. No part of this document may be stored in a retrieval system, transmitted, or otherwise copied or reproduced by any electronic, mechanical, photographic, or recording process without express written permission from Nova Software Labs S.A.R.L.

Published by:



## **Nova Software Labs**

8, rue de Hanovre

75002 Paris, France

Tel: +33 1 58 18 61 61

Fax: +33 1 58 18 61 60

email: [sales@NovaSoftwareLabs.com](mailto:sales@NovaSoftwareLabs.com) or [sales@nsl.fr](mailto:sales@nsl.fr).

Web: <http://www.NovaSoftwareLabs.com> or <http://www.nsl.fr>.

Every precaution has been taken to ensure the accuracy of this document. Nova Software Labs S.A.R.L. assumes no responsibility for errors or omissions, or for damages resulting from the use of this document.

XFaceMaker, XDrawMaker, XnslDrw are registered trademarks of Nova Software Labs S.A.R.L. X Window System is a trademark of the X Consortium. UNIX is a registered trademark of Novell. OSF/Motif and Motif are trademarks of The Open Group, Inc. All trademarks are the property of their respective owners.

Document Number: XFMDDBRefMan v. 7.1.1.

Document date: November 29, 2002

## **Acknowledgments**

The following people have contributed to the development of the XFaceMaker suite:

Alain Bastard, Genevieve Beaujard, Nicole Brion, Alan Char, Eric Durocher, Marc Durocher, France Geze, Eric Herbaut, Elizabeth Huffer, Ion Filotti, Jérôme Maloberti, Sara Sautin, Eric Touchard, Philippe Volle.

Many thanks to all of them and to several others.

---

# Table of Contents

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Introduction . . . . .                              | 1  |
| 1.1. Building Applications . . . . .                   | 2  |
| 1.1.1. XnslTree Alone . . . . .                        | 2  |
| 1.1.2. XnslTree Library and XFaceMaker . . . . .       | 3  |
| 2. The XnslTree Class . . . . .                        | 7  |
| 2.1. Synopsis . . . . .                                | 7  |
| 2.2. Classes . . . . .                                 | 7  |
| 2.3. Description . . . . .                             | 7  |
| 2.3.1. Node Characteristics . . . . .                  | 8  |
| 2.3.2. Edge Characteristics . . . . .                  | 8  |
| 2.3.3. Scrolling . . . . .                             | 9  |
| 2.3.4. Drag and Drop . . . . .                         | 9  |
| 2.3.5. Callbacks . . . . .                             | 10 |
| 2.3.6. Environment Variables . . . . .                 | 10 |
| 2.3.7. Mouse Interactions . . . . .                    | 12 |
| 2.3.8. Reserved Resources . . . . .                    | 12 |
| 2.4. Resources . . . . .                               | 12 |
| 2.4.1. Inherited Resources . . . . .                   | 28 |
| 2.5. Callback Information . . . . .                    | 31 |
| 2.6. Translations . . . . .                            | 31 |
| 2.7. Actions . . . . .                                 | 32 |
| 2.8. Node and Edge Attributes . . . . .                | 34 |
| 2.9. XnslTreePageAttributes Structure . . . . .        | 39 |
| 2.10. File format . . . . .                            | 40 |
| 2.11. Constants . . . . .                              | 41 |
| 2.11.1. Constants for resources . . . . .              | 41 |
| 2.11.2. Constants for attributes . . . . .             | 43 |
| 2.11.3. Constants for the callback structure . . . . . | 43 |
| 3. XnslTree Program Examples . . . . .                 | 45 |
| 3.1. Directory Tree Example . . . . .                  | 45 |
| 3.1.1. User Interaction . . . . .                      | 46 |

---

|                                      |    |
|--------------------------------------|----|
| 3.2. Drag and Drop Example . . . . . | 47 |
| 3.2.1. User Interaction . . . . .    | 49 |
| 3.3. TreeEdit Example . . . . .      | 50 |
| 3.3.1. User Interaction . . . . .    | 52 |
| 4. XnslTree Functions . . . . .      | 55 |
| XnslAddTreeWidgetClass . . . . .     | 56 |
| XnslCreateScrolledTree . . . . .     | 57 |
| XnslTreeCenterNode . . . . .         | 58 |
| XnslTreeComplete . . . . .           | 59 |
| XnslTreeConnectString . . . . .      | 60 |
| XnslTreeCopySelAndSubTrees . . . . . | 61 |
| XnslTreeCreateEdge . . . . .         | 62 |
| XnslTreeCreateNode . . . . .         | 63 |
| XnslTreeCutSelAndSubTrees . . . . .  | 64 |
| XnslTreeDeleteAllEdges . . . . .     | 65 |
| XnslTreeDeleteAllNodes . . . . .     | 66 |
| XnslTreeDeleteEdge . . . . .         | 67 |
| XnslTreeDeleteNode . . . . .         | 68 |
| XnslTreeDeleteNodeList . . . . .     | 69 |
| XnslTreeDeleteSelection . . . . .    | 70 |
| XnslTreeDeleteSubtree . . . . .      | 71 |
| XnslTreeDeselectAllEdges . . . . .   | 72 |
| XnslTreeDeselectAllNodes . . . . .   | 73 |
| XnslTreeEditNode . . . . .           | 74 |
| XnslTreeExpandNode . . . . .         | 75 |
| XnslTreeExpandSelection . . . . .    | 76 |
| XnslTreeGetEdgeAttributes . . . . .  | 77 |
| XnslTreeGetEdgeEnds . . . . .        | 78 |
| XnslTreeGetEdgeFromTo . . . . .      | 79 |
| XnslTreeGetEdges . . . . .           | 80 |
| XnslTreeGetEdgesFrom . . . . .       | 81 |
| XnslTreeGetEdgesTo . . . . .         | 82 |
| XnslTreeGetLeaves . . . . .          | 83 |
| XnslTreeGetNodeAttributes . . . . .  | 84 |
| XnslTreeGetNodes . . . . .           | 85 |
| XnslTreeGetPixmap . . . . .          | 86 |
| XnslTreeGetPixmapMask . . . . .      | 87 |
| XnslTreeGetPredecessors . . . . .    | 88 |

---

|                                            |     |
|--------------------------------------------|-----|
| XnslTreeGetRoots. . . . .                  | 89  |
| XnslTreeGetSelectedEdges. . . . .          | 90  |
| XnslTreeGetSelectedNodes . . . . .         | 91  |
| XnslTreeGetSuccessors. . . . .             | 92  |
| XnslTreeHaveACycle . . . . .               | 93  |
| XnslTreeHighLightNode. . . . .             | 94  |
| XnslTreeInNode . . . . .                   | 95  |
| XnslTreeIsTreeString . . . . .             | 96  |
| XnslTreeLayout. . . . .                    | 97  |
| XnslTreeMatchedNames. . . . .              | 98  |
| XnslTreeMatchedNamesCi . . . . .           | 99  |
| XnslTreeNameToNode . . . . .               | 100 |
| XnslTreeNameToNodeCi . . . . .             | 101 |
| XnslTreeNodeIsVisible. . . . .             | 102 |
| XnslTreeNodeWidget . . . . .               | 103 |
| XnslTreeOnEdge. . . . .                    | 104 |
| XnslTreeReadFile . . . . .                 | 105 |
| XnslTreeReadString . . . . .               | 106 |
| XnslTreeRedisplay . . . . .                | 107 |
| XnslTreeRedisplayAll. . . . .              | 108 |
| XnslTreeReorderPredecessors . . . . .      | 109 |
| XnslTreeReorderSuccessors . . . . .        | 110 |
| XnslTreeSavePsWidget. . . . .              | 111 |
| XnslTreeSelectEdge . . . . .               | 112 |
| XnslTreeSelectionToString. . . . .         | 113 |
| XnslTreeSelectNode . . . . .               | 114 |
| XnslTreeSelectSubTree. . . . .             | 115 |
| XnslTreeSetEdgeAttributes. . . . .         | 116 |
| XnslTreeSetEdgeListAttributes . . . . .    | 117 |
| XnslTreeSetMaskField . . . . .             | 118 |
| XnslTreeSetNodeAttributes . . . . .        | 119 |
| XnslTreeSetNodeListAttributes . . . . .    | 120 |
| XnslTreeSetSubTreeEdgeAttributes . . . . . | 121 |
| XnslTreeSetSubTreeNodeAttributes. . . . .  | 122 |
| XnslTreeSetViewWindowPosition . . . . .    | 123 |
| XnslTreeShrinkNode. . . . .                | 124 |
| XnslTreeShrinkSelection . . . . .          | 125 |
| XnslTreeToString . . . . .                 | 126 |
| XnslTreeUnHighLightNode . . . . .          | 127 |
| XnslTreeUnsetMaskField . . . . .           | 128 |

---

|                         |     |
|-------------------------|-----|
| XnslTreeWriteFile ..... | 129 |
| . Index .....           | 131 |

---

## CHAPTER 1: Introduction

This manual describes the XnslTree widget. The installation procedure for the widget is described in a separate document.

The XnslTree reference pages are in Chapter 2, The XnslTree Class. This is where a developer will find information on the widget's resources, its callbacks and callback structures, the constants it defines, etc. This chapter is organized along the same lines as the Motif widget reference pages.

Chapter 3, XnslTree Program Examples gives tips on how to use the examples provided in the **demos** directory (if the demos have been installed).

The functions that are part of the XnslTree widget library are described in Chapter 4, XnslTree Functions.

The XnslTree widget library distribution includes two copies of the library:

- development library - whenever the widget is instantiated, it requests a license from the NSL network license server. To use this version, you must have a configured license file and the license server daemon, **NSLlicense**, must be running. Please refer to the installation manual for information on configuring and using the NSL license server. Any application using the protected library should free the XnslTree license before exiting. To do so, call the function ***XnslTreeComplete***.
- run-time library - this is delivered in encrypted form. To have access to the unprotected library, you must obtain a library decryption code, once you have purchased the XnslTree widget. Please refer to the installation manual for information on decrypting the library. With the run-time library, the widget can be used independently of any license file or license server, it no longer requests a license. Applications built with this library can run on any binary compatible host. Applications built with the run-time library can continue to call the ***XnslTreeComplete*** function, it will do nothing.

This manual assumes that you are familiar with the X Window System and the Motif widget set and that you have access to the Motif and XtIntrinsics reference manuals.

## 1.1 Building Applications

This section describes what you need to do to build applications that use the Xnsl-Tree library.

You can build an application without using any development tool, just making calls to the XnslTree Library functions, the Xt functions and the Xm functions. Or, if you have XFaceMaker, you can build your interface file with the tool and set the Xnsl-Tree widget resources with the resource editor, then build a compiled or interpreted application.

To include the XnslTree widget in XFaceMaker, you will need to build your customized XFaceMaker executable. To do so, launch **xfm** (or any XFaceMaker executable); in the **File** menu, select **Load Extensions ...** and select the XnslTree extension and any other extensions you want to include in your XFaceMaker executable. Refer to the XFaceMaker manuals for further detail.

### 1.1.1 XnslTree Alone

To build an application directly with the XnslTree Library, you have to

1. Initialize the toolkit: **toplevel = XtAppInitialize( ... )**
2. Write the application and interface code. The interface code will have to instantiate the tree, set resources, include calls to the XnslTree Library functions.
3. If you are using the development library, write an exit function and a Close Handler in which you free the XnslTreeWidget license: **XnslTreeComplete()** This function does nothing when a non-protected run-time library is used so that it is good practice to always call XnslTreeComplete when exiting an application linked with the XnslTree Library.
4. Compile the code and link the object code with any application connected libraries, and the following libraries:

XnslTree, Xm, Xt, X11

If you are using X11R6, you will have to link with additional libraries such as ICE and MRM.

You may need to add some system libraries such as PW, gen, malloc, and so on depending on your environment. You may also have to specify compilation and link flags depending on your platform your compiler, your linker.



Your command line will look something like this:

```
cc -o myprog myprog.c -lXnslTree -lXm -lXt -lX11
```

You must place the Xnsl library before any libraries other than the application specific libraries.

### 1.1.2 XnslTree Library and XFaceMaker

When you use XFaceMaker to build your application, you can choose two execution modes for the interface of your application.

1. Execute the interface in interpreted mode.
2. Execute the interface in compiled mode.

To build such applications you have to use an extended XFaceMaker executable in which you have loaded the XnslTree extension and any other extensions your application may require, e.g. the XRT/Graph widget.

The Makefile generated by a given XFaceMaker executable refers to the include files and the library files of all entities included in the extensions *regardless of whether a particular interface makes any reference to the corresponding widgets or functions*.

For instructions on building a new extended XFaceMaker, refer to the XFaceMaker manuals.

Applications built with an extended XFaceMaker are linked with additional libraries:

**Table 1.1: Basic Extended XFaceMaker Libraries**

| Library   | Link flag   | Description                                                                                                                                 |
|-----------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| XnslStand | -lXnslStand | Common widget support code for XFaceMaker                                                                                                   |
| XnslCtrl  | -lXnslCtrl  | Control widgets: XnslBarGraph, XnslColorBox, XnslColorSB, XnslFontSelector, XnslIndicator, XnslJoystick, XnslMeter, XnslSlider, XnslStepper |
| XnslFm    | -lXnslFm    | XFaceMaker support                                                                                                                          |

- **Interpreted Interface**

Generally, you will use interpreted mode for the interface while you are developing your application: in interpreted mode, changes made in the interface file (.fm file) are taken in account without your having to recompile the interface, or relink the application. This is because, in interpreted mode, the application just loads the .fm file.

To build an application that runs in interpreted mode proceed as follows:

1. With XFaceMaker, build your interface. Set the XnslTree resources as desired.
2. Make sure your interface includes a Quit or Exit button. In the button's activateCallback , call **XnslTreeComplete()** to free the XnslTreeWidget license. You should also add a Close Handler which calls the XnslTreeComplete function.
3. Save your interface as a .fm file, generate the application files using XFaceMaker (Save Appli Files)
4. If your interface calls application functions, add the corresponding code in the xyzApp.c file, or modify the Makefile generated (or the MakeFile pattern) as needed in order to link with the appropriate application code.
5. Compile with the command  

```
make -f xyzMake interpreted
```

where xyz stands for the name of your interface file.

In interpreted mode, the application is linked with the Fm library of XFaceMaker and any libraries required by the extensions with which the extended XFaceMaker you are using was built. Among these, you will have XnslFm, XnslStand and, in our case, XnslTree. The Fm library includes the Ctrl library, so the Ctrl library will not be in your link line, even if your interface includes Control widgets.

### • **Compiled Interface**

If you generate the C code for your interface any changes you make in your interface will require that you rebuild your application. You will generally build applications in compiled mode when your development is well under way or finished, and when you want to deploy your application on a platform where XFaceMaker is not accessible.

To build an application that runs the interface in compiled mode proceed as follows:

1. With XFaceMaker, build your interface. Set the XnslTree resources as desired.
2. Make sure your interface includes a Quit or Exit button. In the button's activateCallback , call **XnslTreeComplete()** to free the

XnslTreeWidget license.

3. If needed, set XFaceMaker's C code generation options.
4. Save your interface as C code, generate the application files using XFaceMaker (Save Appli Files)
5. If your interface calls application functions, add the corresponding code in the xyzApp.c file, or modify the Makefile generated (or the MakeFile pattern) as needed in order to link with the appropriate application code.
6. Compile with the command  
`make -f xyzMake compiled`

where xyz stands for the name of your interface file.

In compiled mode, the application is linked with the Fm\_c library of XFaceMaker and any libraries required by the extensions with which the extended XFaceMaker you are using was built. Among these, you will have XnslStand, XnslCtrl and, in our case, XnslTree.



## CHAPTER 2: The XnslTree Class

### 2.1 Synopsis

*#include <Xnsl/Tree.h>*

### 2.2 Classes

The **XnslTree** class inherits resources and behavior from the **Core**, **Composite**, **Constraint**, and **XmManager** classes

The class pointer is **xnslTreeWidgetClass**.

The class name is **XnslTree**.

### 2.3 Description

The XnslTree widget is designed to handle the editing and display of tree graphs. A tree is composed of nodes and edges (sometimes called links). A node is displayed as an image and/or a label. An edge is a line that connects two nodes. Nodes and edges have classical graphical attributes like foreground, background, etc... The widget is able to layout a tree. That means that the widget is able to compute the coordinates of nodes and edges to provide a readable tree display.

Three layout types are implemented:

- m **XnslISI** (Information Science Institute algorithm) This is the most basic tree layout algorithm.
- m **XnslCTG** ( Compact Tree Grapher algorithm) . A much more compact layout.
- m **XnslORTHOGONAL**. A layout algorithm that looks like a Windows® 95 Explorer window.

When large trees are to be displayed, the widget can be placed in a ScrolledWindow with the scrollingPolicy resource set to *application\_defined*. The widget will handle its own scrolling, and will load only the data that lie within the viewing window.

A complete view of a large tree can be obtained by switching the widget to overview mode. In this mode, the entire tree is displayed, nodes being represented by rectangles and edges by segments. This mode does not allow any interactive editing of the tree, except drag and drop operations. A rectangle materializing the zone that would be displayed in the extended view is overlaid on the tree overview. The user can move the rectangle over the tree to select the area to display (scroll to) when exiting from overview mode.

Nodes and edges can be selected for editing, using the standard Motif selection with mouse button 1 click (multiple selection with Control button 1) or by rubber-band selection (any object which lies entirely within the rubber-band is selected). Node or edge selection triggers the `treeSelectionCallback` which passes to the application all the information necessary to edit selected nodes and/or edges.

Nodes are not widgets, their attributes are handled directly within the tree widget. However, the tree widget being a composite widget it will, in future releases, be able to handle widget nodes in addition to its own pre-defined nodes. This early release does not manage the geometry of any widget children.

Trees are saved in files using a pre-defined format, described in “File format” on page 40.

### 2.3.1 Node Characteristics

A pixmap and a character string (the node name, displayed in the label) can be specified for each node. The position of the label relative to the pixmap can be specified: the label can be to the left, to the right, above or below the pixmap. The widget handles interactive editing of a node’s name by overlaying an text input widget on the node label, in which the user may enter a new character string. The action which overlays the text widget on a node is button1 click in a selected node’s label, with no node motion. The `treeChangeNameCallback` is triggered when input is completed, allowing the application to confirm the change. By default, interactive node name editing is disabled. See the `XnslnNodeEditMode` resource.

The following graphic attributes are available to decorate nodes:

- m font
- m pixmap or bitmap image
- m label
- m foreground color
- m background color

### 2.3.2 Edge Characteristics

Edges are segments which interconnect two nodes. Arrows can be drawn at the ends of an edge. An edge can have a label (display its name) and/or a pixmap, which is placed in the middle of the segment.

The following graphic attributes are available to decorate edges:

- m font

- m label/pixmap
- m foreground color
- m background color
- m dashes
- m arrows
- m line width

### 2.3.3 Scrolling

The XnslTree widget handles its own scrolling when it is placed within an XmScrolledWindow whose scrollingPolicy resource is set to XmAPPLICATION\_DEFINED. In this mode, the size of the tree widget is controlled by the size of the viewing window and only the visible portion of the tree is loaded in the widget. As the window is scrolled, the portion of the tree corresponding to the position of the scrollbars is loaded by the widget. The function XnslCreateScrolledTree creates a tree widget within an XmScrolledWindow with application defined scrolling. This works in much the same way as the XmText widget in an XmScrolledWindow, created using the XmCreateScrolledText function.

It is also possible for the application to control scrolling of the tree. The widget provides the XnslNtreeXOffset and XnslNtreeYOffset resources and the function XnslTreeSetViewWindowPosition for this purpose.

### 2.3.4 Drag and Drop

The XnslTree widget implements the Motif drag and drop protocol. The user can copy selected portions of the tree between XnslTree widgets or between XnslTree widgets and other widget classes that implement this protocol. The XnslTree widget is registered as a drag source for target ***String*** and as a drop site for targets ***String*** and ***CompoundText***. The data dragged is the *selection*.

Drag and drop operations are controlled by callbacks and resources. These can be used by the application to cancel or authorize drag and drop actions; they can be used to process data transferred to/from the XnslTree widget.

The XnslTree widget implements its own drag-under effects as enabled by the auto-Highlight resource: during a drag and drop operation, the nodes are highlighted as a visual aid to the user to inform him that, as he drops, he will drop over the node that is highlighted.

### 2.3.5 Callbacks

The widget supports the callbacks listed below. The callback structure returned is the same for all the callbacks. The callback descriptions indicate which of the structure fields are valid for a given callback.

- m **treeActivateCallback**: triggered by double clicking on a node or an edge. The node or edge on which the event occurred is passed.
- m **treeChangeNameCallback**: triggered by the interactive edition of a node's name. Lets the application accept, with or without modification, or reject the newly specified name.
- m **treeDeleteCallback**: triggered when a node or an edge is deleted so that the application can free any memory it has allocated in connection with the edge or node being deleted, in particular user\_data attribute information.
- m **treeDragCallback**: triggered when a drag operation is initiated.
- m **treeDropCallback**: triggered when a drop operation is initiated.
- m **treeInputCallback**: triggered when the widget receives a keyboard or mouse button event. The node or edge on which the event occurred is returned, if any.
- m **treeLoadUserDataCallback**: triggered for each node or edge created when a tree or sub-tree in XnslTree file format is being loaded. The application can use this callback to modify the node's or edge's user\_data attribute 'on the fly'.
- m **treeMoveCallback**: triggered when the selection is moved.
- m **treeMoveVerifyCallback**: triggered when the selection is moved, before the treeMoveCallback is called. This lets the application accept or reject the new position of the selection. If the motion is accepted, the treeMoveCallback is triggered.
- m **treeSaveUserDataCallback**: triggered for each node or edge written when a tree or sub-tree is being saved in XnslTree file format. The application can use this callback to modify the node's or edge's user\_data attribute 'on the fly'.
- m **treeSelectionCallback**: triggered when a selection is modified. The list of selected nodes and edges is passed.

### 2.3.6 Environment Variables

The Xpm library used by the widget requires an **rgb.txt** file. To find it, the widget first looks for the **XNSL\_TREE\_RGB\_DB** environment variable. If this is defined, the widget tries to open the file specified in the variable. If this variable is not set, the widget tries to open **/usr/lib/X11/rgb.txt** or, on Solaris, **/usr/openwin/lib/X11/**



**rgb.txt**. If the file cannot be opened, the widget will scream. Set **NSL\_TREE\_RGB\_DB** to **\$NSLHOME/lib/X11/treergb.txt**. This file is installed by the XnsITree widget installation script.

Remember to provide a color description file and to set the variable if necessary, also when you deploy any applications that make use of the XnsITree widget.

To locate pixmaps and bitmaps, the widget uses the **XBMLANGPATH** environment variable. This is a standard X environment variable that is used to specify the directories searched to find bitmap files. **%B** is used as the substitution character for the bitmap file name

**2.3.7 Mouse Interactions**

| Mouse Button Action                     | Result                                                                                                                                                                                                              |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Button1 click on an edge or a node      | select edge or node.                                                                                                                                                                                                |
| Button1 click on a selected node name   | Overlay a text or textField over the node to edit its name.                                                                                                                                                         |
| Button 1 double click anywhere          | invoke the treeActivateCallback                                                                                                                                                                                     |
| Ctrl Button1 click on an edge or a node | adjust edge or node selection.                                                                                                                                                                                      |
| Button1 motion                          | if started outside any node or edge: clear selection and do rubberband selection.<br>if started on selection and if allowed (not frozen): drag selection with cursor.<br>in overview mode, move overview rectangle. |
| Ctrl Button1 motion                     | do rubberband selection and add it to existing selection.                                                                                                                                                           |
| Button2 press                           | initiate Motif drag operation.                                                                                                                                                                                      |
| Button2 motion                          | Motif drag.                                                                                                                                                                                                         |
| Button 2 release                        | initiate Motif drop operation.                                                                                                                                                                                      |
| Button3 press                           | Change cursor to hand cursor                                                                                                                                                                                        |
| Button3 motion                          | HandMove: scroll tree with mouse                                                                                                                                                                                    |
| Button3 release                         | Change cursor to normal cursor                                                                                                                                                                                      |

**2.3.8 Reserved Resources**

Some resources and resource values have been defined for future developments. These are marked with a †. Do not attempt to set them, since you would be accessing functionality that is not fully implemented in this release.

**2.4 Resources**

The **XnslTree** widget class defines the new set of resources listed in the table below.

C indicates that the resource can be set at creation time;

S indicates that the resource can be set using XtSetValues;

G indicates that the resource can be retrieved using XtGetValues;

### XnslTree Resource Set

| Name<br>Class                                              | Default<br>Type                 | Access |
|------------------------------------------------------------|---------------------------------|--------|
| XnslNtreeActivateCallback<br>XnslCTreeActivateCallback     | NULL<br>XtRCallback             | C      |
| XnslNtreeArrowHeight<br>XnslCTreeArrowHeight               | 15<br>XmRShort                  | CSG    |
| XnslNtreeArrowWidth<br>XnslCTreeArrowWidth                 | 5<br>XmRShort                   | CSG    |
| XnslNtreeArrows<br>XnslCTreeArrows                         | XnslNO_ARROW<br>XnslRTreeArrows | CSG    |
| XnslNtreeAutoHighlight<br>XnslCTreeAutoHighlight           | True<br>XmRBoolean              | CSG    |
| XnslNtreeAutoLayout<br>XnslCTreeAutoLayout                 | True<br>XmRBoolean              | CSG    |
| XnslNtreeAutomaticMode<br>XnslCTreeAutomaticModet          | False<br>XmRBoolean             | CSG    |
| XnslNtreeBorder<br>XnslCTreeBorder                         | 4<br>XmRDimension               | CSG    |
| XnslNtreeChangeNameCallback<br>XnslCTreeChangeNameCallback | NULL<br>XtRCallback             | C      |
| XnslNtreeCheckValidEdge<br>XnslCTreeCheckValidEdge         | False<br>XmRBoolean             | CSG    |
| XnslNtreeDeleteCallback<br>XnslCtreeDeleteCallback         | NULL<br>XtRCallback             | C      |
| XnslNtreeDoHandMove<br>XnslCtreeDoHandMove                 | True<br>XmRBoolean              | CSG    |
| XnslNtreeDoRubberband<br>XnslCtreeDoRubberband             | True<br>XmRBoolean              | CSG    |
| XnslNtreeDragCallback<br>XnslCtreeDragCallback             | NULL<br>XtRCallback             | C      |

**XnslTree Resource Set**

| Name<br>Class                                                  | Default<br>Type                            | Access |
|----------------------------------------------------------------|--------------------------------------------|--------|
| XnslNtreeDropCallback<br>XnslCTreeDropCallback                 | NULL<br>XtRCallback                        | C      |
| XnslNtreeEdgeFont<br>XnslCTreeEdgeFont                         | “fixed”<br>XmRString                       | CSG    |
| XnslNtreeEdgeRatioAutoCompute<br>XnslCTreeEdgeRatioAutoCompute | False<br>XmRBoolean                        | CSG    |
| XnslNtreeFile<br>XnslCTreeFile                                 | NULL<br>XnslRTreeFile                      | CSG    |
| XnslNtreeFillArrow<br>XnslCTreeFillArrow                       | True<br>XmRBoolean                         | CSG    |
| XnslNtreeHeight<br>XnslCTreeHeight                             | 1<br>XnslRTreeDimension                    | G      |
| XnslNtreeInputCallback<br>XnslCTreeInputCallback               | NULL<br>XtRCallback                        | C      |
| XnslNtreeLayoutAlgorithm<br>XnslCTreeLayoutAlgorithm           | XnslCTG<br>XnslRTreeLayoutAlgorithm        | CSG    |
| XnslNtreeLoadUserDataCallback<br>XnslCTreeLoadUserDataCallback | NULL<br>XtRCallback                        | C      |
| XnslNtreeMargin<br>XnslCTreeMargin                             | 4<br>XmRDimension                          | CSG    |
| XnslNtreeMinusPixmap<br>XnslCTreeMinusPixmap                   | NULL<br>XmRString                          | CSG    |
| XnslNtreeMoveCallback<br>XnslCTreeMoveCallback                 | NULL<br>XtRCallback                        | C      |
| XnslNtreeMoveVerifyCallback<br>XnslCTreeMoveVerifyCallback     | NULL<br>XtRCallback                        | C      |
| XnslNtreeNodeAltPixmap<br>XnslCTreeNodeAltPixmap               | NULL<br>XmRString                          | CSG    |
| XnslNtreeNodeAltPixmapMask<br>XnslCTreeNodeAltPixmapMask       | NULL<br>XmRString                          | CSG    |
| XnslNtreeNodeDisplayPolicy<br>XnslCTreeNodeDisplayPolicy       | XnslSHOW_ALL<br>XnslRTreeNodeDisplayPolicy | CSG    |
| XnslNtreeNodeEditMode<br>XnslCTreeNodeEditMode                 | XnslNONE<br>XnslRTreeNodeEditMode          | CSG    |

## XnslTree Resource Set

| Name<br>Class                                                      | Default<br>Type                                         | Access |
|--------------------------------------------------------------------|---------------------------------------------------------|--------|
| XnslNtreeNodeFont<br>XnslCTreeNodeFont                             | “fixed”<br>XmRString                                    | CSG    |
| XnslNtreeNodeHeight<br>XnslCTreeNodeHeight                         | 0<br>XmRDimension                                       | CSG    |
| XnslNtreeNodePixmap<br>XnslCTreeNodePixmap                         | NULL<br>XmRString                                       | CSG    |
| XnslNtreeNodePixmapMask<br>XnslCTreeNodePixmapMask                 | NULL<br>XmRString                                       | CSG    |
| XnslNtreeNodeWidth<br>XnslCTreeNodeWidth                           | 0<br>XmRDimension                                       | CSG    |
| XnslNtreeNormalCursor<br>XnslCTreeNormalCursor                     | NULL<br>XtRCursor                                       | CSG    |
| XnslNtreeOrientation<br>XnslCTreeOrientation                       | XnslLEFT_RIGHT<br>XnslRTreeOrientation                  | CSG    |
| XnslNtreeOverview<br>XnslCTreeOverview                             | False<br>XmRBoolean                                     | CSG    |
| XnslNtreeOverviewMargin<br>XnslCTreeOverviewMargin                 | 3<br>XmRInt                                             | CSG    |
| XnslNtreePixmapLabelSep<br>XnslCTreePixmapLabelSep                 | 2<br>XmRInt                                             | CSG    |
| XnslNtreePlusPixmap<br>XnslCTreePlusPixmap                         | NULL<br>XmRString                                       | CSG    |
| XnslNtreeRedisplayMethod<br>XnslCTreeRedisplayMethod               | Xnsl_REDISELAY_ON_TOPLEVELI<br>XnslRTreeRedisplayMethod | CSG    |
| XnslNtreeResizeMethod<br>XnslCTreeResizeMethod                     | Xnsl_RESIZE_ON_TOPLEVELI<br>XnslRTreeResizeMethod       | CSG    |
| XnslNtreeSaveUserDataCallback<br>XnslCTreeSaveUserDataCallback     | NULL<br>XtRCallback                                     | C      |
| XnslNtreeScrollbarDisplayPolicy<br>XnslCTreeScrollbarDisplayPolicy | XnslSTATIC<br>XnslRTreeScrollbarDisplayPolicy           | CSG    |
| XnslNtreeSelectionCallback<br>XnslCTreeSelectionCallback           | NULL<br>XtRCallback                                     | C      |
| XnslNtreeSpace<br>XnslCTreeSpace                                   | 50<br>XmRDimension                                      | CSG    |

**XnslTree Resource Set**

| Name<br>Class                              | Default<br>Type           | Access |
|--------------------------------------------|---------------------------|--------|
| XnslNtreeType †<br>XnslCTreeType           | XnslTREE<br>XnslRTreeType | CSG    |
| XnslNtreeValue<br>XnslCTreeValue           | NULL<br>XmRString         | CSG    |
| XnslNtreeWaitCursor<br>XnslCTreeWaitCursor | “watch”<br>XtRCursor      | CSG    |
| XnslNtreeWidth<br>XnslCTreeWidth           | 1<br>XnslRTreeDimension   | G      |
| XnslNtreeXOffset<br>XnslCTreeXOffset       | 0<br>XmRInt               | CSG    |
| XnslNtreeYOffset<br>XnslCTreeYOffset       | 0<br>XmRInt               | CSG    |

**XnslNtreeActivateCallback**

This callback is triggered by a double click anywhere in the widget. The following fields of the XnslTreeCallbackStruct are valid:

- m reason is XnslCR\_TREE\_ACTIVATE
- m event is the event which triggered the callback
- m node is the ID of the node that was activated if any.
- m node\_detail if event occurred in a node.
- m edge is the ID of the edge that was activated if any.
- m edge\_detail if event occurred in an edge.

**XnslNtreeArrowHeight**

Specifies the height in pixels of arrows in the widget.

**XnslNtreeArrowWidth**

Specifies the width in pixels of arrows in the widget.

**XnslNtreeArrows**

Specifies default arrows for the widget; this may be overridden using the edge attributes. Possible values are:

m XnsINO\_ARROW  
m XnsIFROM\_ARROW  
m XnsITO\_ARROW  
m XnsIFROM\_TO\_ARROW

### **XnsINtreeAutoHighlight**

If True, Nodes are highlighted during a drag and drop operation.

### **XnsINtreeAutoLayout**

If True, specifies that layout is to be recomputed each time a resource that has an incidence on layout results is changed. If False, layout is recomputed only when the XnsINtreeLayout function is called.

### **XnsINtreeAutomaticMode**

If True, the node under the cursor is automatically shrunk or expanded when the activateCallback is triggered, unless the *doit* field of the callback structure is set to False. When a node is shrunk ( it was not in the shrunk state), the edge pixmap is automatically changed to the pixmap specified in the XnsINtreePlusPixmap resource. When a node is expanded (it was in the shrunk state when the event occurred), the associated edge pixmap is changed to the one specified in the XnsINtreeMinusPixmap resource.

If True, and a node is selected, its pixmap is changed to the one specified in the XnsINtreeNodeAltPixmap resource, if any, and its pixmap mask is changed to the one specified in the XnsINtreeNodeAltPixmapMask resource, if any.

If False, the activateCallback does not affect the shrunk state of a node, nor does node selection affect the node pixmap.

### **XnsINtreeBorder**

Specifies the size in pixels of the border around each node. This controls the spacing between nodes during layout.

### **XnsINtreeChangeNameCallback**

This callback is triggered before and after a node name is changed interactively. The reason field of the callback structure is used to distinguish the two phases.

m reason is XnsICR\_TREE\_CHANGE\_NAME\_VERIFY: the user has entered a new character string, which is in the *name* field. The event field is set to NULL. The application can change the content of the *name* field before it validates the name change by setting the *doit* field to True. If *doit* is set to False, the name change is not done.

m reason is `XnslCR_TREE_NAME_CHANGED`: the name change has been done.

### **XnslNtreeCheckValidEdge**

If True, specifies that the widget should verify that an edge is valid before creating it. If False, no verification of edges is performed at creation time, which makes the procedure much faster. An edge is considered not valid if:

- m the source and destination are the same
- m an edge between this source and this destination already exists
- m the edge generates a node with several parents
- m the edge induces a cycle

### **XnslNtreeDeleteCallback**

This callback is triggered when a node or an edge is deleted. It can be used by the application to free any memory allocated specifically for the node or edge being deleted. The following fields of the `XnslTreeCallbackStruct` are valid:

- m reason is `XnslCR_TREE_DELETE`
- m event is set to `NULL`
- m node is the ID of the node being deleted, or `NULL` if an edge is being deleted.
- m edge is the ID of the edge being deleted, or `NULL` if a node is being deleted.

### **XnslNtreeDoHandMove**

If True, enable scrolling of the `XnslTree` viewing area within the scrolled window using `HandMove`.

### **XnslNtreeDoRubberband**

If True, enable rubberband selection.

### **XnslNtreeDragCallback**

This callback is invoked before and after a drag operation is initiated. When it is invoked before the operation, the reason is set to `XnslCR_TREE_DRAG_VERIFY`. In this case the application can cancel the drag operation by setting the *doit* field of the `XnslTreeCallbackStruct` to false. When it is invoked after the operation, the reason is set to `XnslCR_TREE_DRAG`. The *widget* field of the `XnslTreeCallbackStruct` is set to the drag context returned by the `XmStartDrag` function.

### **XnslNtreeDropCallback**



This callback is invoked during a drop operation. The callback is called at different times during the drop process. The reason of the `XnslTreeCallbackStruct` passed is used to distinguish the various steps:

- m `XnslCR_TREE_DROP_ENTER`: The callback is called with the *reason* set to `XnslCR_TREE_DROP_ENTER` when a drop operation enters the widget. The *widget* field is set to the drag context widget.
- m `XnslCR_TREE_DROP_LEAVE`: The callback is called with the *reason* set to `XnslCR_TREE_DROP_LEAVE` when a drop operation leaves the widget. The *widget* field is set to the drag context widget.
- m `XnslCR_TREE_DROP_MOTION`: The callback is called with the *reason* set to `XnslCR_TREE_DROP_MOTION` when a drop operation moves inside the widget. The *widget* field is set to the drag context widget.
- m `XnslCR_TREE_DROP_STRING`: The callback is called with the *reason* set to `XnslCR_TREE_DROP_STRING` when a string is dropped in the widget. The *value* field is set to the string being dropped.
- m `XnslCR_TREE_DROP_CTEXT`: The callback is called with the reason set to `XnslCR_TREE_DROP_CTEXT` when a compound text is dropped in the widget. The *value* field is set to the compound string being dropped.
- m `XnslCR_TREE_DROP_VERIFY`: The callback is called with the reason set to `XnslCR_TREE_DROP_VERIFY` before the drop operation occurs. The application can cancel the drop by setting the *doit* field of the `XnslTreeCallbackStruct` to false. The *operation* field is set to `XmDROP_MOVE` or `XmDROP_COPY` as appropriate, or to `XmDROP_NOOP` if *dropAction* is not `XmDROP`.

In all these cases the *node* and *edge* field of the `XnslTreeCallbackStruct` are set to the node or edge that is under the pointer.

### **XnslNtreeEdgeFont**

The name of the font used for edge labels if none is specified in the edge attributes.

### **XnslNtreeEdgeRatioAutoCompute**

If true, calculate the position of the edge attachment point according to the cardinal direction of the associated node. If false, the position of the edge attachment point is the center of the node except if otherwise specified in the edge

attributes. Edge attribute attachment specifications override this resource.

### **XnslNtreeFile**

The name of the file loaded in the widget.

### **XnslNtreeFillArrow**

If True, arrows are filled. If false, arrows are outlined.

### **XnslNtreeHeight**

Height of the tree in pixels. This is a read only resource.

### **XnslNtreeInputCallback**

This callback is triggered by the following events:

- KeyPress
- KeyRelease
- ButtonPress
- ButtonRelease
- PointerMotion
- PointerMotionHint
- ButtonMotion

The following fields of the XnslTreeCallbackStruct are valid:

m reason is XnslCR\_TREE\_INPUT

m event is the event which triggered the callback

m node is the ID of the node under the pointer for the event, if any

m node\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the node

m edge is the ID of the edge under the pointer for the event, if any

m edge\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the edge

### **XnslNtreeLayoutAlgorithm**

Specifies the layout algorithm used to display the tree. Acceptable values are:

m XnslCTG ( Compact Tree Grapher algorithm)

m XnslISI (Information Science Institute algorithm)

m XnslORTHOGONAL

## XnslNtreeLoadUserDataCallback

This callback is triggered for each node or edge created when a tree or sub-tree in XnslTree file format is being loaded into the widget. The callback is triggered immediately after the node/edge is created, to let the application change the `user_data` and `user_data_type` attributes. If no callback procedure is attached, the `user_data` and `user_data_type` attributes are those read for the node/edge in the String loaded.

The following fields of the `XnslTreeCallbackStruct` are valid when the callback is invoked for a node:

m reason is `XnslCR_LOAD_NODE_USER_DATA`

m widget is the `XnslTree` widget ID

m node is the ID of the node being created

m value\_type is the data type of the `user_data`. Acceptable values are `XnslINT_USER_DATA` and `XnslSTRING_USER_DATA`. This field is initialized to the `value_type` read in the string representation of the node (default is `XnslINT_USER_DATA`) and can be changed by the application.

m value is initialized to the value of `user_data` read in the string representation of the node, according to `value_type`:

- `XnslINT_USER_DATA`, value is set to `atoi(value_string)`;
- `XnslSTRING_USER_DATA`, value is set to `<value_string>`, the string loaded.

Value can be modified by the application.

m doit specifies whether the node's `user_data` should be modified. If `doit` is set to 1 (the default) the node's `user_data` is set to `value`, and its `user_data_type` is set to `value_type`. If `doit` is set to 0, `value` and `value_type` are ignored, the node's `user_data` attribute is left unchanged.

The following fields of the `XnslTreeCallbackStruct` are valid when the callback is invoked for an edge:

m reason is `XnslCR_LOAD_EDGE_USER_DATA`

m widget is the `XnslTree` widget ID

m edge is the ID of the edge being created

m value\_type is the data type of the `user_data`. Acceptable values are `XnslINT_USER_DATA` and `XnslSTRING_USER_DATA`. This field is initialized to the `value_type` read in the string representation of the edge (default is `XnslINT_USER_DATA`) and can be changed by the application.

m value is initialized to the value of user\_data read in the string representation of the edge, according to value\_type:

- XnslINT\_USER\_DATA, value is set to `atoi(value_string)`;
- XnslSTRING\_USER\_DATA, value is set to `<value_string>`, the string loaded.

Value can be modified by the application.

m doit specifies whether the edge's user\_data should be modified. If doit is set to 1 (the default) the edge's user\_data is set to *value*, and its user\_data\_type is set to *value\_type*. If doit is set to 0, *value* and *value\_type* are ignored, the edge's user\_data attribute is left unchanged.

### **XnslNtreeMargin**

Specifies the tree margin in pixels. This specifies the minimum distance between any nodes and the border of the widget, vertically and horizontally. This resource does not apply when the tree is in overview mode. XnslNtreeOverviewMargin is used instead.

### **XnslNtreeMinusPixmap**

Specifies the name of the pixmap that is added to edges in the expanded state.

### **XnslNtreeMoveCallback**

This callback is triggered when a node is moved interactively. The following fields of the XnslTreeCallbackStruct are valid:

m reason is XnslCR\_TREE\_MOVE

m event is the event which triggered the callback

m node is the ID of the node under the pointer for the event, if any

m node\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the node

m edge is the ID of the edge under the pointer for the event, if any

m edge\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the edge

m node\_count is the number of nodes in the selection

m nodes is the list of node ID's in the selection

m edge\_count is the number of edges in the selection

m edges is the list of edge ID's in the selection

**XnslNtreeMoveVerifyCallback**

This callback is triggered before a node is moved in interactive node motion. This callback is called before the new position is effective; it lets the application forbid the motion by means of the *doit* structure field.

m reason is XnslCR\_TREE\_MOVE\_VERIFY

m event is the event which triggered the callback

m node is the ID of the node under the pointer for the event, if any

m node\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the node

m edge is the ID of the edge under the pointer for the event, if any

m edge\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the edge

m node\_count is the number of nodes in the selection

m nodes is the list of node ID's in the selection

m edge\_count is the number of edges in the selection

m edges is the list of edge ID's in the selection

m doit is True by default. Set it to False to prevent the motion: the XnslNtree-MoveCallback will not be called.

**XnslNtreeNodeAltPixmap**

Specifies the pixmap name used for selected nodes if none is specified in the node attributes and the XnslNtreeAutomaticMode resource is True.

**XnslNtreeNodeAltPixmapMask**

Specifies the bitmap name used to mask the node pixmaps of selected nodes if no alternate pixmap mask is specified in the node attributes and the XnslNtree-AutomaticMode resource is True.

**XnslNtreeNodeDisplayPolicy**

Specifies if the tree displays node labels and/or pixmaps; accepted values are:

m XnslSHOW\_ALL node pixmaps and labels are both displayed

m XnslSHOW\_LABELS node labels only are displayed

m XnslSHOW\_PIXMAPS node pixmaps only are displayed

m XnslSHOW\_NONE nothing is displayed.

### **XnslNtreeNodeEditMode**

Specifies if node names are editable, and, if so, if they are single line or multiple line names. Acceptable values are:

m XnslNONE

m XnslSINGLE\_LINE

m XnslMULTIPLE\_LINE

If node names are editable, an input text widget is overlayed on the node label when the user clicks on the node label of a selected node. The user can type a new node name in this field. In single line mode, the Return key validates the name. In multiple line mode, the Enter key (usually on the numeric pad) validates the name, the Return key generating a new line. When the name is validated, the input field disappears.

### **XnslNtreeNodeFont**

The name of the font used to display node labels if none is specified in the node attributes.

### **XnslNtreeNodeHeight**

If not 0, specifies the height in pixels that is used for all nodes in the tree, for layout calculations.

### **XnslNtreeNodePixmap**

Specifies the pixmap name used for nodes if none is specified in the node attributes.

### **XnslNtreeNodePixmapMask**

Specifies the bitmap name used to mask the node pixmaps if no pixmap mask is specified in the node attributes.

### **XnslNtreeNodeWidth**

If not 0, specifies the width in pixels that is used for all nodes in the tree, for layout calculations. The distance between nodes at different levels of the hierarchy still depends on the XnslNtreeSpace resource, in addition to node width.

### **XnslNtreeNormalCursor**

Specifies the name of the cursor used when no interaction is being processed. Default value is 0, i.e. the default cursor.

### **XnslNtreeOrientation**

Specifies the orientation of the tree display; accepted values are:

m XnslLEFT\_RIGHT  
m XnslRIGHT\_LEFT  
m XnslTOP\_BOTTOM  
m XnslBOTTOM\_TOP

### **XnslNtreeOverview**

If True, display the tree in overview mode, i.e. display the entire tree, nodes being shown as simple rectangles. The tree is not editable in overview mode. A rectangle representing the normal viewing window area is overlaid on the tree; this can be used to select the area to display in normal mode (XnslNtreeOverview False).

### **XnslNtreeOverviewMargin**

Specifies the tree margin in pixels in overview mode. This specifies the minimum distance between any overview rectangles and the border of the widget, vertically and horizontally. The margin is filled with the foreground color.

### **XnslNtreePixmapLabelSep**

Specifies, in pixels, the separation between the pixmap and the label associated to a node.

### **XnslNtreePlusPixmapName**

Specifies the name of the pixmap that is added to edges in the shrunk state.

### **XnslNtreeRedisplayMethod**

Specify how redisplay is handled. If set to Xnsl\_REDISPLAY\_ON\_TOPLEVEL, the widget builds a list of modified rectangles as it processes modification requests. It triggers the redisplay of the rectangles in its list when the application returns to the main event loop. If set to Xnsl\_REDISPLAY\_SYSTEMATIC the widget redisplay itself immediately on any modification that potentially changes its aspect. If set to Xnsl\_REDISPLAY\_APPLICATION, the widget is redisplayed only when the application calls the XnslTreeRedisplay function.

### **XnslNtreeResizeMethod**

Specifies how tree resizing is handled. If set to Xnsl\_RESIZE\_ON\_TOPLEVEL, the widget recomputes its size when the application returns to the main event loop. If set to Xnsl\_RESIZE\_SYSTEMATIC the widget recomputes its size immediately on any modification that potentially changes its size.

### **XnslNtreeSaveUserDataCallback**

This callback is triggered for each node or edge saved when a tree or sub-tree is being saved in XnslTree file format. The callback is triggered immediately before the node/edge is saved, to let the application change the `user_data` and `user_data_type` attributes written in the file. If no callback procedure is attached, the `user_data` and `user_data_type` attributes are written in the file as is.

The following fields of the `XnslTreeCallbackStruct` are valid for a node:

`m reason` is `XnslCR_SAVE_NODE_USER_DATA`

`m widget` is the XnslTree widget ID

`m node` is the ID of the node being saved

`m value_type` is the data type of the `user_data`. Acceptable values are `XnslINT_USER_DATA` and `XnslSTRING_USER_DATA`. This field is used by the widget to save the value field.

- `XnslINT_USER_DATA`, value is saved as `%d` in the string representation of the node.
- `XnslSTRING_USER_DATA`, value is saved as `%s` in the string representation of the node.

If value is not in agreement with `value_type`, the widget may crash.

`m value` is initialized to the node's `user_data` attribute. It can be modified by the application.

`m doit` specifies whether the node's `user_data` should be saved. If `doit` is set to 1 (the default) the value and `value_type` are written to the file in the `user_data` and `user_data_type` fields. If it is set to 0, `user_data` is not saved in the file.

The following fields of the `XnslTreeCallbackStruct` are valid for an edge:

`m reason` is `XnslCR_SAVE_EDGE_USER_DATA`

`m widget` is the XnslTree widget ID

`m edge` is the ID of the edge being saved

`m value_type` is the data type of the `user_data`. Acceptable values are `XnslINT_USER_DATA` and `XnslSTRING_USER_DATA`. This field is used by the widget to save the value field.

- `XnslINT_USER_DATA`, value is saved as `%d` in the string representation of the edge.
- `XnslSTRING_USER_DATA`, value is saved as `%s` in the string representation of the edge.



If value is not in agreement with value\_type, the widget may crash.

m value is initialized to the edge's user\_data attribute. It can be modified by the application.

m doit specifies whether the edge's user\_data should be saved. If doit is set to 1 (the default) the value and value\_type are written to the file in the user\_data and user\_data\_type fields. If it is set to 0, user\_data is not saved in the file.

### **XnsINScrollbarDisplayPolicy**

Specifies how to display the scrollbars of the scrolledWindow in which the XnsITree widget has been placed; accepted values are:

m XnsISTATIC            the scrollbars are always displayed.

m XnsIAS\_NEEDED        the scrollbars are displayed only when needed.

### **XnsINtreeSelectionCallback**

This callback is triggered when the selection is changed. The following fields of the XnsITreeCallbackStruct are valid:

m reason is XnsICR\_TREE\_SELECTION

m event is the event which triggered the callback.

m node is the ID of the node whose selection status is changed. It is set to NULL if no node is involved, and in rubberband selection.

m node\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the node.

m edge is the ID of the edge whose selection status is changed. It is set to NULL if no edge is involved, and in rubberband selection.

m edge\_detail indicates if the event occurred in the label, in the pixmap or nowhere in the edge.

m node\_count is the number of nodes in the selection.

m nodes is the list of node ID's in the selection.

m edge\_count is the number of edges in the selection.

m edges is the list of edge ID's in the selection.

### **XnsINtreeSpace**

Specifies the distance in pixels between nodes at different levels of the hierarchy. This controls the extension of edges.

### **XnsINtreeType †**

Specifies the type of graph. Reserved for future use. The only possible value is:  
m XnslTREE

**XnslNtreeValue**

Specifies the content of the tree. This is a string whose format is the one used to save/read a tree file (see “File format” on page 40). Reading this resource gives the content of the tree. Setting this resource concatenates the string specified with the existing content of the tree.

**XnslNtreeWaitCursor**

Specifies the name of the cursor used when the widget is busy. Default value is “watch”.

**XnslNtreeWidth**

Width of the tree in pixels. This is a read only resource.

**XnslNtreeXOffset**

Specifies the value of the horizontal scrollBar if the widget is in a scrolledWindow.

**XnslNtreeYOffset**

Specifies the value of the vertical scrollBar if the widget is in a scrolledWindow.

**2.4.1 Inherited Resources**

**XnslTree** inherits the following resources from the named superclass. For a description of each resource, refer to the man page of the superclass.

**XmManager Resource Set**

| Name<br>Class                                  | Default<br>Type                | Access |
|------------------------------------------------|--------------------------------|--------|
| XmNbottomShadowColor<br>XmCBottomShadowColor   | dynamic<br>Pixel               | CSG    |
| XmNbottomShadowPixmap<br>XmCBottomShadowPixmap | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG    |
| XmNforeground<br>XmCForeground                 | dynamic<br>Pixel               | CSG    |
| XmNhelpCallback<br>XmCCallback                 | NULL<br>XtCallbackList         | C      |

## XmManager Resource Set

| Name<br>Class                            | Default<br>Type                 | Access |
|------------------------------------------|---------------------------------|--------|
| XmNhighlightColor<br>XmCHighlightColor   | dynamic<br>Pixel                | CSG    |
| XmNhighlightPixmap<br>XmCHighlightPixmap | dynamic<br>Pixmap               | CSG    |
| XmNinitialFocus<br>XmCInitialFocus       | dynamic<br>Widget               | CSG    |
| XmNnavigationType<br>XmCNavigationType   | XmTAB_GROUP<br>XmNavigationType | CSG    |
| XmNshadowThickness<br>XmCShadowThickness | dynamic<br>Dimension            | CSG    |
| XmNstringDirection<br>XmCStringDirection | dynamic<br>XmStringDirection    | CG     |
| XmNtopShadowColor<br>XmCTopShadowColor   | dynamic<br>Pixel                | CSG    |
| XmNtopShadowPixmap<br>XmCTopShadowPixmap | dynamic<br>Pixmap               | CSG    |
| XmNtraversalOn<br>XmCTraversalOn         | True<br>Boolean                 | CSG    |
| XmNunitType<br>XmCUnitType               | dynamic<br>unsigned char        | CSG    |
| XmNuserData<br>XmCUserData               | NULL<br>XtPointer               | CSG    |

## Composite Resource Set

| Name<br>Class                          | Default<br>Type     | Access |
|----------------------------------------|---------------------|--------|
| XmNchildren<br>XmCReadOnly             | NULL<br>WidgetList  | G      |
| XmNinsertPosition<br>XmCInsertPosition | NULL<br>XtOrderProc | CSG    |
| XmNnumChildren<br>XmCReadOnly          | 0<br>Cardinal       | G      |

Core Resource Set

| Name<br>Class                                                  | Default<br>Type                | Access |
|----------------------------------------------------------------|--------------------------------|--------|
| XmNaccelerators<br>XmCAccelerators                             | dynamic<br>XtAccelerators      | N/A    |
| XmNancestorSensitive<br>XmCSensitive                           | dynamic<br>Boolean             | G      |
| XmNbackground<br>XmCBackground                                 | dynamic<br>Pixel               | CSG    |
| XmNbackgroundPixmap<br>XmCPixmap                               | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG    |
| XmNborderColor<br>XmCBorderColor                               | XtDefaultForeground<br>Pixel   | CSG    |
| XmNborderPixmap<br>XmCPixmap                                   | XmUNSPECIFIED_PIXMAP<br>Pixmap | CSG    |
| XmNborderWidth<br>XmCBorderWidth                               | 0<br>Dimension                 | CSG    |
| XmNcolormap<br>XmCColormap                                     | dynamic<br>Colormap            | CG     |
| XmNdepth<br>XmCDepth                                           | dynamic<br>int                 | CG     |
| XmNdestroyCallback<br>XmCCallback                              | NULL<br>XtCallbackList         | C      |
| XmNheight<br>XmCHeight                                         | dynamic<br>Dimension           | CSG    |
| XmNinitialResourcesPersistent<br>XmCInitialResourcesPersistent | True<br>Boolean                | C      |
| XmNmappedWhenManaged<br>XmCMappedWhenManaged                   | True<br>Boolean                | CSG    |
| XmNscreen<br>XmCScreen                                         | dynamic<br>Screen              | CG     |
| XmNsensitive<br>XmCSensitive                                   | True<br>Boolean                | CSG    |
| XmNtranslations<br>XmCTranslations                             | dynamic<br>XtTranslations      | CSG    |
| XmNwidth<br>XmCWidth                                           | dynamic<br>Dimension           | CSG    |

Core Resource Set

| Name<br>Class       | Default<br>Type | Access |
|---------------------|-----------------|--------|
| XmNx<br>XmCPosition | 0<br>Position   | CSG    |
| XmNy<br>XmCPosition | 0<br>Position   | CSG    |

2.5 Callback Information

The callback structure returned is the same for all callbacks of the tree widget. The application should first look at the reason field to determine which fields of the structure are valid.

```
typedef struct {
    int                reason;
    XEvent             *event;
    int                x;
    int                y;
    Widget             widget;
    XnslTreeNodeId     node;
    XnslTreeDetail      node_detail;
    XnslTreeEdgeId     edge;
    XnslTreeDetail      edge_detail;
    XnslTreeNodeId     *nodes;
    int                node_count;
    XnslTreeEdgeId     *edges;
    int                edge_count;
    String             name;
    XnslTreeUserDataTy value_type;
    XtPointer           value;
    Boolean            doit;
    unsigned char       operation;
} XnslTreeCallbackStruct;
```

2.6 Translations

The following translations are defined for the tree widget.

```
Ctrl <Btn1Down>, Ctrl <Btn1Motion>(1+):
    ModMotion()
```

```

Ctrl <Btn1Down>, Ctrl <Btn1Motion>(1+), <Btn1Up> :
    ModDisarm()
Ctrl <Btn1Down>, Ctrl <Btn1Motion>(1+), Ctrl <Btn1Up>:
    ModDisarm()
Ctrl <Btn1Up>:
    ModDisarm()
Ctrl <Btn1Down>:
    ModArm()
<Btn1Down>, <Btn1Motion>(1+):
    Motion()
<Btn1Down>, <Btn1Motion>(1+), <Btn1Up>:
    Disarm()
<Btn1Down>(2):
    Activate()
<Btn1Down>, <Btn1Up>:
    Disarm()
<Btn1Down>:
    Arm()
<Btn2Down>:
    StartDrag()
<Btn3Down>:
    HandArm()
<Btn3Down>, <Btn3Motion>(1+):
    HandMove()
<Btn3Down>, <Btn3Motion>(1+), <Btn3Up>:
    HandDisarm()
<Btn3Down>, <Btn3Up>:
    HandDisarm()

```

## 2.7 Actions

The following actions are defined for the tree widget.

m *Activate()*

- If the event occurred on an edge or on a node invoke the **activateCallback**. Otherwise, do nothing.

m *Arm()*

- In overview mode, center the viewing window rectangle on the cursor position.
- In standard mode, if the event occurred on a node or on an edge, invoke the **selectionCallback**. In addition, if the event is on a node, prepare selection drag rectangle.

- In standard mode, if the event is not on a node or edge, deselect any existing selection, initialize the rubberband selection, invoke the **selectionCallback**.

m *Disarm()*

- In standard mode (not overview, not rubberband), if there has been motion (the selection has been moved) call the **moveVerifyCallback**. If it returns with *doit* True, make the motion effective and invoke the **moveCallback**.
- In standard mode, if there has been no motion and the event has occurred on a selected label, overlay a text input widget on the label if **XnslNtreeNodeEditMode** is not set to *XnslNone*.
- If rubberband selection has been initialized, finish rubberband selection: select all nodes and edges which lie fully within the rubberband rectangle, invoke the **selectionCallback**.

m *ModArm()*

- If the event occurred on an edge or a node, toggle the selection and invoke the **selectionCallback**.
- If the event is not on a node or edge, initialize rubberband extend selection.

m *ModDisarm()*

- If rubberband extend selection has been initialized, finish rubberband extend selection: select all nodes and edges which lie fully within the rubberband rectangle, adding them to the existing selection, invoke the **selectionCallback**.

m *ModMotion()*

- If rubberband extend selection has been initialized, draw the rubberband.

m *Motion*

- In standard mode, and if selection drag has been initiated, draw selection drag rectangle(s) at current position.
- If rubberband selection has been initialized, draw the rubberband.

m *StartDrag*

- Initiate drag and drop operation.

m *HandArm*

- Change cursor to hand cursor.

m *HandMove*

- Scroll tree in viewing window.

### m *HandDisarm*

- Change cursor to normal cursor.

Caution: most of the actions described above are interrelated. Beware of undesired side effects when redefining the translation table.

## 2.8 Node and Edge Attributes

Nodes and edges can be created or read, their attributes can be changed, by means of functions which let the application access the attribute structures.

### m **XnslTreeNodeAttributes:**

- alt\_pixmap - **String** - Specifies the pixmap name used for the node when it is selected and the XnslNtreeAutomaticMode resource is True.
- alt\_pixmap\_mask - **String** - Specifies the bitmap name used to mask the node alternate pixmap for the node when it is selected and the XnslNtreeAutomaticMode resource is True.
- background - **String** - the name of the background color. If no background attribute is specified when a node is created, the background resource of the widget is used.
- font - **String** - the name of the font used in the label. If no font attribute is specified when a node is created, the treeLabelFont resource of the widget is used at creation.
- foreground - **String** - the name of the foreground color. If no foreground attribute is specified when a node is created, the foreground resource of the widget is used.
- frozen - **Boolean** - False if the node's position can be changed interactively. If no frozen attribute is specified when a node is created, it is set to False.
- height - **Dimension** - the height, in pixels, of the node. This is a read only attribute, the height of a node being determined by the height of its name, of its pixmap, and by their relative position.
- label\_position - **XnslTreeLabelPosition** - the position of the label relative to the image. If no label\_position is specified when a node is created, the value is XnslTreeLabelBottom. Accepted values are:
  - XnslLABEL\_LEFT
  - XnslLABEL\_RIGHT
  - XnslLABEL\_TOP



- `XnslLABEL_BOTTOM`
- `XnslLABEL_CENTER`
- `layout` - **Boolean** - True if the node is included in the layout calculation. If no `layout` attribute is specified when a node is created, it is set to True.
- `mapped` - **Boolean** - True if the node is mapped. Any edges or nodes connected to an unmapped node are not mapped. Unmapped nodes are not included in the layout calculations. If no `mapped` attribute is specified when a node is created, it is set to True.
- `name` - **String** - the node name; the name is displayed in the label. If no `name` attribute is specified when a node is created, it is set to NULL.
- `pixmap` - **String** - the name of the image (xbm or xpm format). If no `pixmap` attribute is specified when a node is created, it is set to NULL.
- `pixmap_mask` - **String** - The name of the pixmap mask applied to the node pixmap. This mask can be used to remove a pixmap's underlying rectangle. If no `pixmap` attribute is specified when a node is created, it is set to NULL.
- `selectable` - **Boolean** - If False, the node cannot be selected. If no `selectable` attribute is specified when a node is created, it is set to True.
- `selected` - **Boolean** - True if the node is selected. If no `selected` attribute is specified when a node is created, it is set to False.
- `shrunk` - **Boolean** - True if the node is shrunk. If no `shrunk` attribute is specified when a node is created, it is set to False.
- `user_data` - **XtPointer** - a pointer to user information. Lets the user attach a pointer to any information that is to be associated to the node. If no `user_data` attribute is specified when a node is created, it is set to NULL.
- `user_data_type` - **XnslTreeUserDataTypes** - the type of data stored in the `user_data` attribute. Acceptable values are: `XnslINT_USER_DATA` (the `user_data` is saved as %d in the node's string representation) and `XnslSTRING_USER_DATA` (the `user_data` is saved as %s in the node's string representation). The application can use the `XnslSaveUserDataCallback` and the `XnslLoadUserDataCallback` to change the node's `user_data` and `user_data_type` on the fly.
- `width` - **Dimension** - the width, in pixels, of the node. This is a read only attribute, the width of a node being determined by the size of its name, of its pixmap, and by their relative position.
- `x` - **XnslTreePosition** - the x coordinate in pixels of the upper left corner of the node, relative to the viewing port. If no `x` attribute is specified when a node is created, it is set to 0.

- **y** - **XnslTreePosition** - the y coordinate in pixels of the upper left corner of the node, relative to the viewing port. If no y attribute is specified when a node is created, it is set to 0.

m **XnslTreeEdgeAttributes:**

- **arrows** - **XnslTreeEdgeArrows** - specifies whether arrows should be drawn at the ends of the edge. If no arrows attribute is specified when an edge is created, the value is set to XnslTreeDefaultArrow and the treeArrows resource of the widget is used. Accepted values are:
  - XnslDEFAULT\_ARROW
  - XnslNO\_ARROW
  - XnslFROM\_ARROW
  - XnslTO\_ARROW
  - XnslFROM\_TO\_ARROW
- **background** - **String** - the name of the background color. If no background attribute is specified when an edge is created, the background resource of the widget is used.
- **dashes** - **char** - the size, in pixels, of alternating foreground and background color line fragments used to draw an edge. If it is 0, the edge is drawn as a solid line. If no dashes attribute is specified when an edge is created, it is set to 0.
- **font** - **String** - the name of the font used in the label. If no font attribute is specified when an edge is created, the treeEdgeFont resource of the widget is used.
- **foreground** - **String** - the name of the foreground color. If no foreground attribute is specified when an edge is created, the foreground resource of the widget is used.
- **from** - **XnslTreeNodeId** - the ID of the node which is the starting point of the edge, or source.
- **fxoffset** - **int** - specifies the horizontal offset in pixels of the edge's attachment point relative to the attachment point specified by the fxratio value. If fxoffset is not specified when an edge is created, it is set to 0.
- **fxratio** - **int** - specifies the horizontal position of the edge's attachment point relative to the middle of the source node. Values lie between -100 (the left-most point) and +100 (the right-most point), 0 being the middle. If fxratio is not specified when an edge is created, it is set to 0.
- **fyoffset** - **int** - specifies the vertical offset in pixels of the edge's attachment

point relative to the attachment point specified by the `fyratio` value. If `fyoffset` is not specified when an edge is created, it is set to 0.

- `fyratio` - **int** - specifies the vertical position of the edge's attachment point relative to the middle of the source node. Values lie between -100 (the top-most point) and +100 (the bottom-most point), 0 being the middle. If `fyratio` is not specified when an edge is created, it is set to 0.
- `line_width` - **int** - width, in pixels, of the line used to draw the edge. If `line_width` is not specified when an edge is created, it is set to 0.
- `mapped` - **Boolean** - True if the edge is mapped. If `mapped` is not specified when an edge is created, it is set to True.
- `name` - **String** - the edge name; if the label is displayed, the name is its contents. If both a label and a pixmap are specified, the pixmap is displayed over the label. If `name` is not specified when an edge is created, it is set to NULL.
- `pixmap` - **String** - the name of the image (xbm or xpm format) associated to the edge. If both a label and a pixmap are specified, the pixmap is displayed over the label. If `pixmap` is not specified when an edge is created, it is set to NULL.
- `selectable` - **Boolean** - If False, the edge cannot be selected. If `selectable` is not specified when an edge is created, it is set to True.
- `selected` - **Boolean** - True if the edge is selected, False if the edge is not selected or not selectable. This attribute cannot be set to True if the `selectable` attribute is not True.
- `to` - **XnsId** - the ID of the node which is the ending point of the edge or source.
- `txoffset` - **int** - specifies the horizontal offset in pixels of the edge's attachment point relative to the attachment point specified by the `txratio` value. If `txoffset` is not specified when an edge is created, it is set to 0.
- `txratio` - **int** - same as `fxratio` but for the destination node.
- `tyoffset` - **int** - specifies the vertical offset in pixels of the edge's attachment point relative to the attachment point specified by the `tyratio` value. If `tyoffset` is not specified when an edge is created, it is set to 0.
- `tyratio` - **int** - same as `fyratio` but for the destination node.
- `user_data` - **XtPointer** - a pointer to user information. Lets the user attach a pointer to any information that is to be associated to the edge. If no `user_data` attribute is specified when an edge is created, it is set to NULL.

- `user_data_type` - **XnslTreeUserDataType** - the type of data stored in the `user_data` attribute. Acceptable values are: `XnslINT_USER_DATA` (the `user_data` is saved as `%d` in the edge's string representation) and `XnslSTRING_USER_DATA` (the `user_data` is saved as `%s` in the edge's string representation). The application can use the `XnslSaveUserDataCallback` and the `XnslLoadUserDataCallback` to change the edge's `user_data` and `user_data_type` on the fly.

The functions which use these structures also have a mask argument which lets the application specify which fields of the structure are valid. To specify several attributes, specify the OR of the mask fields.

The mask fields and the corresponding attribute name for the `XnslTreeNodeAttributes` are:

|                                            |                                |
|--------------------------------------------|--------------------------------|
| <code>XnslTreeNodeAltPixmapMask</code>     | <code>"alt_pixmap"</code>      |
| <code>XnslTreeNodeAltPixmapMaskMask</code> | <code>"alt_pixmap_mask"</code> |
| <code>XnslTreeNodeBackgroundMask</code>    | <code>"background"</code>      |
| <code>XnslTreeNodeFontMask</code>          | <code>"font"</code>            |
| <code>XnslTreeNodeForegroundMask</code>    | <code>"foreground"</code>      |
| <code>XnslTreeNodeFrozenMask</code>        | <code>"frozen"</code>          |
| <code>XnslTreeNodeHeightMask</code>        | <code>"height"</code>          |
| <code>XnslTreeNodeLabelPositionMask</code> | <code>"label_position"</code>  |
| <code>XnslTreeNodeLayoutMask</code>        | <code>"layout"</code>          |
| <code>XnslTreeNodeMappedMask</code>        | <code>"mapped"</code>          |
| <code>XnslTreeNodeNameMask</code>          | <code>"name"</code>            |
| <code>XnslTreeNodePixmapMask</code>        | <code>"pixmap"</code>          |
| <code>XnslTreeNodePixmapMaskMask</code>    | <code>"pixmap_mask"</code>     |
| <code>XnslTreeNodeSelectableMask</code>    | <code>"selectable"</code>      |
| <code>XnslTreeNodeSelectedMask</code>      | <code>"selected"</code>        |
| <code>XnslTreeNodeShrunkMask</code>        | <code>"shrunk"</code>          |
| <code>XnslTreeNodeUserDataMask</code>      | <code>"user_data"</code>       |
| <code>XnslTreeNodeUserDataMaskMask</code>  | <code>"user_data_mask"</code>  |
| <code>XnslTreeNodeWidthMask</code>         | <code>"width"</code>           |
| <code>XnslTreeNodeXMask</code>             | <code>"x"</code>               |
| <code>XnslTreeNodeYMask</code>             | <code>"y"</code>               |

The mask fields and the corresponding attribute name for the `XnslTreeEdgeAttributes` are:

|                                         |                           |
|-----------------------------------------|---------------------------|
| <code>XnslTreeEdgeArrowsMask</code>     | <code>"arrows"</code>     |
| <code>XnslTreeEdgeBackgroundMask</code> | <code>"background"</code> |
| <code>XnslTreeEdgeDashesMask</code>     | <code>"dashes"</code>     |

|                              |                  |
|------------------------------|------------------|
| XnslTreeEdgeFontMask         | "font"           |
| XnslTreeEdgeForegroundMask   | "foreground"     |
| XnslTreeEdgeFromMask         | "from"           |
| XnslTreeEdgeFxoffsetMask     | "fxoffset"       |
| XnslTreeEdgeFxratioMask      | "fxratio"        |
| XnslTreeEdgeFyoffsetMask     | "fyoffset"       |
| XnslTreeEdgeFyratioMask      | "fyratio"        |
| XnslTreeEdgeLineWidthMask    | "line_width"     |
| XnslTreeEdgeMappedMask       | "mapped"         |
| XnslTreeEdgeNameMask         | "name"           |
| XnslTreeEdgePixmapMask       | "pixmap"         |
| XnslTreeEdgeSelectableMask   | "selectable"     |
| XnslTreeEdgeSelectedMask     | "selected"       |
| XnslTreeEdgeToMask           | "to"             |
| XnslTreeEdgeTxoffsetMask     | "txoffset"       |
| XnslTreeEdgeTxratioMask      | "txratio"        |
| XnslTreeEdgeTyoffsetMask     | "tyoffset"       |
| XnslTreeEdgeTyratioMask      | "tyratio"        |
| XnslTreeEdgeUserDataMask     | "user_data"      |
| XnslTreeEdgeUserDataTypeMask | "user_data_type" |

## 2.9 XnslTreePageAttributes Structure

```
typedef struct{
char          *name;
float         width;
float         height;
float         v_margin;
float         h_margin;
int           orientation;
int           rows;
int           columns;
Boolean       fit;
unsigned int  resolution;
} XnslTreePageAttributes;
```

- **name** - The media type name passed to the printer, e.g. A4, Letter, B5, Executive, etc..
- **width** - The width of the page in inches.
- **height** - The height of the page in inches.

- **v\_margin** - The vertical margin in inches. This is usually set to 0.24 inches.
- **h\_margin** - The horizontal margin in inches. This is usually set to 0.24 inches.
- **orientation** - The Postscript page orientation: this can be `XnslTREE_LANDSCAPE` or `XnslTREE_PORTRAIT`.
- **rows** - The number of pages, vertically, on which the output is to be generated. Set this to -1 to obtain the number of rows corresponding to the drawing. If fit is *True*, this must be set to 1.
- **columns** - The number of pages, horizontally, on which the output is to be generated. Set this to -1 to obtain the number of columns corresponding to the drawing. If fit is *True*, this must be set to 1.
- **fit** - If *True*, make the output fit on a single page. The setting of **rows** and **columns** must be 1 if **fit** is set to *True*.
- **resolution** - The resolution of the output, in dpi (dots per inch). If this is set to 0, the screen resolution is used to calculate the postscript output.

When printing the XnslTree widget, specify all the parameters of the **Xnsl-TreePageAttributes** structure.

## 2.10 File format

When a tree is saved into a file, the following format is used:

First, the nodes list is saved, each node having a unique identification. Then, the edges list is saved; all edges must have a source and a destination node ID.

```

file:          nodes_list edges_list
nodes_list:    nothing |
               node nodes_list
node:          node <id>
               x                [int]
               y                [int]
               <name>           [string]
               <foreground>     [string]
               <background>     [string]
               <font>           [string]
               <pixmap>         [string]
               <alt_pixmap>     [string]
               <mask>           [string]
               <alt_mask>       [string]
               <label_position> [int]
               <mapped>         [int]
```

```

        <selected>          [int]
        <selectable>       [int]
        <shrunk>           [int]
        <laidout>          [int]
        <frozen>           [int]
        <user_data_type>   int|string
        <user_data>       [int]||[string]
; \n
edge_list: nothing |
           edge edges_list
edge:      edge
           from          [int]
           to            [int]
           <name>        [string]
           <foreground>  [string]
           <background>  [string]
           <pixmap>      [string]
           <line_width>   [int]
           <dashes>       [int]
           <fxratio>      [int]
           <fyratio>      [int]
           <txratio>      [int]
           <tyratio>      [int]
           <fxoffset>     [int]
           <fyoffset>     [int]
           <txoffset>     [int]
           <tyoffset>     [int]
           <arrows>       [int]
           <mapped>       [int]
           <selected>     [int]
           <selectable>   [int]
           <font>         [string]
           <user_data_type> int|string
           <user_data>   [int]||[string]
; \n

```

## 2.11 Constants

The values for the constants defined for this widget are listed below:

### 2.11.1 Constants for resources

- XnslTreeEdgeArrows:
  - XnslDEFAULT\_ARROW

- XnslNO\_ARROW
- XnslFROM\_ARROW
- XnslTO\_ARROW
- XnslFROM\_TO\_ARROW
- XnslTreeLayoutAlgorithm:
  - XnslCTG (Compact Tree Grapher algorithm)
  - XnslISI (Information Science Institute algorithm)
  - XnslORTHOGONAL
- XnslTreeNodeDisplayPolicy:
  - XnslSHOW\_ALL
  - XnslSHOW\_LABELS
  - XnslSHOW\_PIXMAPS
  - XnslSHOW\_NONE
- XnslTreeNodeEditMode:
  - XnslNONE
  - XnslSINGLE\_LINE
  - XnslMULTIPLE\_LINE
- XnslTreeOrientation:
  - XnslLEFT\_RIGHT
  - XnslRIGHT\_LEFT
  - XnslTOP\_BOTTOM
  - XnslBOTTOM\_TOP
- XnslTreePageAttributes Orientation:
  - XnslTREE\_LANDSCAPE
  - XnslTREE\_PORTRAIT
- XnslTreeRedisplayMethod:
  - Xnsl\_REDISPLAY\_SYSTEMATIC
  - Xnsl\_REDISPLAY\_APPLICATION
  - Xnsl\_REDISPLAY\_ON\_TOPLEVEL
- XnslTreeResizeMethod:
  - Xnsl\_RESIZE\_SYSTEMATIC



- Xnsl\_RESIZE\_APPLICATION<sup>1</sup>
- Xnsl\_RESIZE\_ON\_TOPLEVEL
- XnslTreeScrollbarDisplayPolicy:
  - XnslSTATIC
  - XnslAS\_NEEDED
- XnslTreeType:
  - XnslTREE

### **2.11.2 Constants for attributes**

- XnslTreeEdgeArrows:
  - XnslDEFAULT\_ARROW
  - XnslNO\_ARROW
  - XnslFROM\_ARROW
  - XnslTO\_ARROW
  - XnslFROM\_TO\_ARROW
- XnslTreeLabelPosition:
  - XnslLABEL\_LEFT
  - XnslLABEL\_RIGHT
  - XnslLABEL\_TOP
  - XnslLABEL\_BOTTOM
  - XnslLABEL\_CENTER
- XnslTreeUserDataTypes:
  - XnslINT\_USER\_DATA
  - XnslSTRING\_USER\_DATA

### **2.11.3 Constants for the callback structure**

- XnslTreeDetail:
  - XnslNOWHERE
  - XnslIN\_LABEL
  - XnslIN\_PIXMAP

---

1. Do not set the resource to this value, as the widget would never recompute its size.

- Callback reasons :
  - XnslCR\_TREE\_ACTIVATE
  - XnslCR\_TREE\_CHANGE\_NAME\_VERIFY
  - XnslCR\_TREE\_NAME\_CHANGED
  - XnslCR\_TREE\_DELETE
  - XnslCR\_TREE\_DRAG
  - XnslCR\_TREE\_DRAG\_VERIFY
  - XnslCR\_TREE\_DROP\_CTEXT
  - XnslCR\_TREE\_DROP\_ENTER
  - XnslCR\_TREE\_DROP\_LEAVE
  - XnslCR\_TREE\_DROP\_MOTION
  - XnslCR\_TREE\_DROP\_STRING
  - XnslCR\_TREE\_DROP\_VERIFY
  - XnslCR\_TREE\_INPUT
  - XnslCR\_TREE\_LOAD\_EDGE\_USER\_DATA
  - XnslCR\_TREE\_LOAD\_NODE\_USER\_DATA
  - XnslCR\_TREE\_MOVE
  - XnslCR\_TREE\_MOVE\_VERIFY
  - XnslCR\_TREE\_SAVE\_EDGE\_USER\_DATA
  - XnslCR\_TREE\_SAVE\_NODE\_USER\_DATA
  - XnslCR\_TREE\_SELECTION
- Drop operation:
  - XmDROP\_NOOP
  - XmDROP\_MOVE
  - XmDROP\_COPY

## CHAPTER 3: XnslTree Program Examples

### 3.1 Directory Tree Example

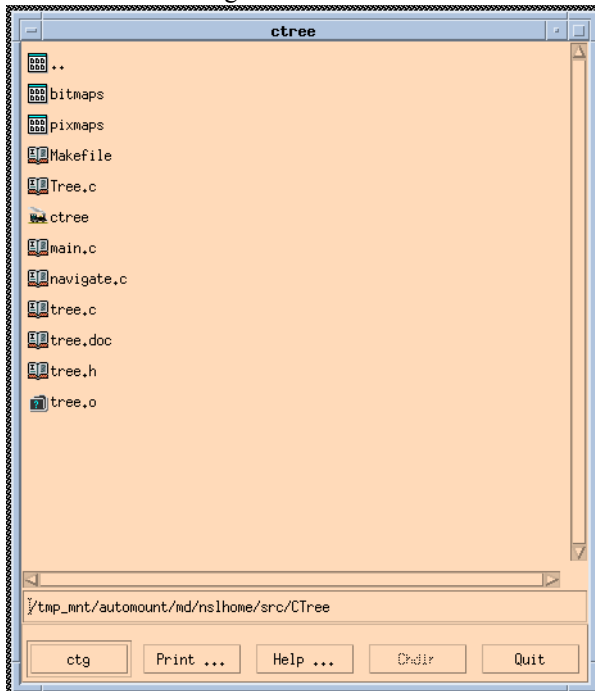
This example is designed to illustrate the basic operation of the XnslTree widget. It is built without any GUI builder, with direct C calls to Motif and X functions. To launch this example, change to the directory

```
$NSLHOME/demos/xnsltree/ctree
```

and launch the executable

```
./ctree
```

Your screen should look something like this:



This application is built around a Form which includes a **scrolledTree**, a text output area, and some pushButtons at the bottom of the window.

The source code for this demo is available in the CTree directory and may be used as desired to build similar applications. It includes **Tree.c**, the main program, **navigate.c** and **tree.c**. **Tree.c** instantiates the widgets and specifies the callbacks, while **navigate.c** has the code to navigate in directories and build the directory tree with node attributes that change according to the type of file being represented. The type of files recognized are directory files, image files (.xpm, .gif, ...), text files, executable files. **tree.c** implements some of the behavior of the application. A Makefile is provided to help you rebuild the demo. However, the original executable is built with an unprotected XnslTree library. If you rebuild the demo and link with a protected XnslTree library, the demo will request a license when you launch it.

### 3.1.1 User Interaction

When it is first launched, **ctree** displays the directory tree of the current directory. For each node, the node name is the name of the file, the node pixmap differs according to the type of file.

7. **Expanding a directory** - To expand a directory, e.g. *bitmaps*, double click on a directory node (do not attempt to expand directory '..' as this will do a *chdir* to the directory above, it will not expand). When a directory contains sub-directories (see below, navigating the directory tree) it is identified by a small '+' pixmap associated to the edge; this changes to a '-' pixmap when you expand the directory. You can expand directories by clicking `button1` on that pixmap, instead of double clicking on the node.
8. **Navigating the directory tree** - When you select a directory (`button1` click on directory node), the **Chdir** button at the bottom of the window becomes sensitive. Click on it, and you will change to the directory you selected. You can also climb the directory tree by double clicking `button1` on the node '..'. If you climb the directory tree, you will eventually find directories that have sub-directories: a small icon is associated to the edge. Notice that the current directory name is displayed in the text output field above the buttons.
9. **Changing the layout** - When you launch this application, the `treeLayout` resource is set to *XnslORTHOGONAL*. You can switch to *XnslCTG* by clicking on the button labeled **ctg** in the lower left of the window. The button label changes to **isi**. If you click on it, you will switch to *XnslISI* and the button label will change to **orthogonal**. Try all three modes, with a tree where the nodes are expanded so as to get a good feel for the differences between the layout options.
10. **Hand move** - If you press `button3` on the widget, the cursor changes to the hand cursor. If the view window is smaller than the tree, i.e. the scroll bar sliders are smaller than the troughs, you can scroll the tree in the view window by drag-

ging the hand.

11. ***Saving the tree in a Postscript file*** - If you click on the ***Print...*** button, you will pop a file selector. Specify where you want your .ps file created. The tree is copied into the Postscript file, with as many A4 pages as necessary. If your printer does not accept A4 paper size, you can view the .ps file with ghostscript, or you can change the source code to specify a different page size.
12. ***Getting Help*** - A ***Help*** button opens a dialog box in which this text is displayed.
13. ***Exiting*** - Click on the ***Quit*** button to exit the demo.

### 3.2 Drag and Drop Example

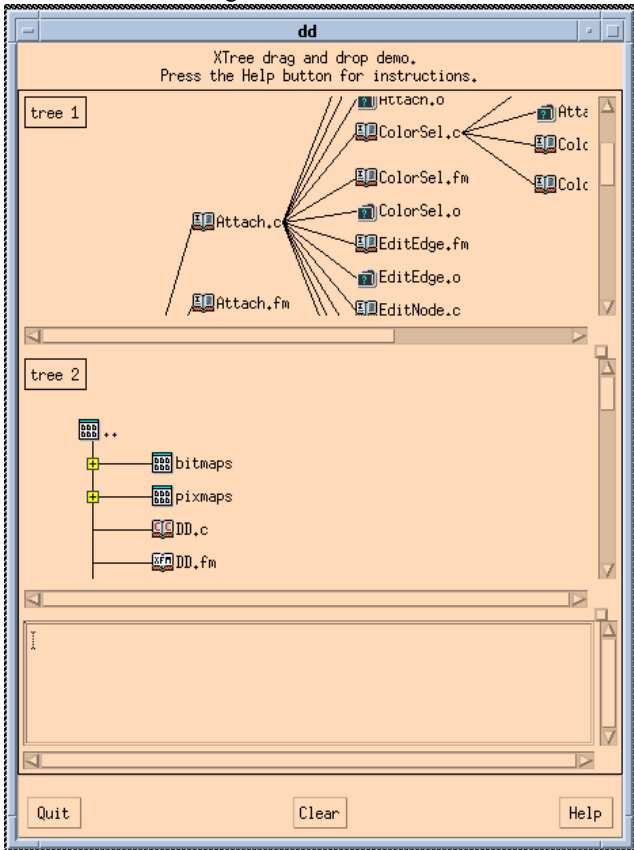
This example is designed to illustrate drag and drop operations as they are implemented in the XnsITree widget. The example is built with XFaceMaker. To launch the demo, change to the directory

`$NSLHOME/demos/xnsltree/draganddrop`

and launch the executable

`./dad`

Your screen should look something like this:



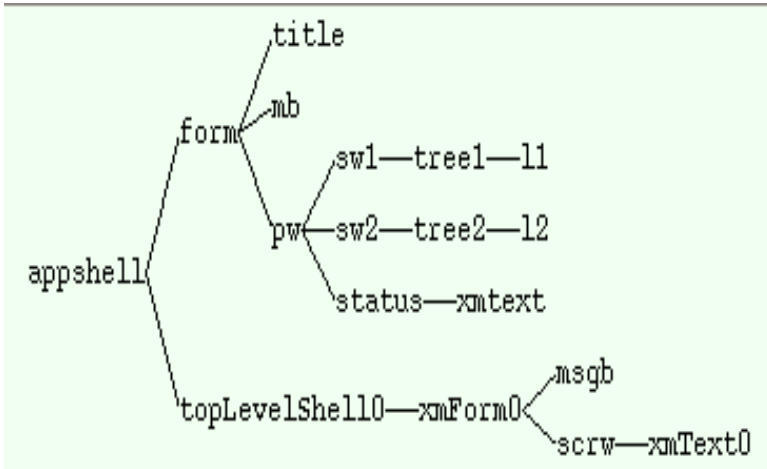
The application is built around a `panedWindow` with three panes; the two upper panes contain an **XnslTree** widget in a `scrolledWindow`. The bottom pane contains a `scrolledText` which will receive messages as the user performs drag and drop operations in the two **XnslTree** widgets. The **XnslTree** widgets are identified as **tree1** and **tree2**.

Buttons at the bottom of the window let you *Quit* the demo, *Clear* the `scrolledText` area, pop a `messageBox` for *Help*.

The source code for this demo is provided along with the executable. All resources are specified using `XFaceMaker`; callbacks are written using the `FACE` language, with, occasionally, a call to an application function. Resource settings and callback scripts are in the file **DAD.fm**; application functions are in the file **dad.c**. A `Makefile`

is provided to help you rebuild the demo. However, the original executable is built with an unprotected XnsITree library. If you rebuild the demo and link with a protected XnsITree library, the demo will request a license when you launch it.

The widget tree for the interface is:



### 3.2.1 User Interaction

In addition to standard XnsITree behavior, such as node or edge selection and button 3 scrolling with the hand cursor this application implements node shrink/expand, node name change, overview mode, and XnsITree drag and drop.

1. **shrink/expand** - To shrink a node's sub-tree, or expand a shrunk node, double click button 1, on a node's pixmap or on a non-selected node name. When a node is shrunk, the string ' ->' is appended to its name. When it is expanded, the string ' ->' is removed from its name. A node can be marked as shrunk even if it has no subtree.
2. **node name change** - To change a node's name, select a node and button1 click on its name. Type a new name in the textfield overlayed on the node name and validate with Return. The demo adds a "\*" character at the beginning of the name, to show that the name was modified. The tree is layouted after the name change.
3. **overview mode** - To toggle one of the tree widgets to/from overview mode, type the letter 'o' in it. Make sure that the widget has the keyboard focus. A rectangle representing the normal mode viewing window is overlayed on the tree widget in overview mode. Drag the rectangle ( button1 drag) or move it with the scroll bars to specify the view window's position.

4. **drag and drop** - To drag a node from one of the widgets, press button2 on the node. The node and its sub-tree, if any, are highlighted. As long as the drag cursor is in one of the widgets, the foreground color is *blue* in tree1 and *red* in tree2. This color change serves as an indication that a drag and drop operation is under way in the widget.
  - m When the drag cursor is moved over a node, it is highlighted, as a cue that, if you drop, you will drop in the node. If you drop in a node, the sub-tree being dragged is connected to the node. If you drop in the widget outside any node, the sub-tree is added at the root of the tree. Any time you drop valid data (in this case, the target is String and the content of the string is a tree description) in one of the widgets, the attributes are updated and the tree is laidout. You can drag and drop data between the 2 widgets, or within tree1; tree2 rejects drag and drop to itself.
  - m Instead of drag and dropping tree description strings, you can drag and drop **compoundText**, e.g. taken from a pushButton label. If you drop in a node in tree1, the node name is changed to the string extracted from the compound text. If you drop outside any node in tree1, the drop is accepted but the string is not copied anywhere. If you drop in tree2, a new node is created with the string as its label, and no pixmap. If you drop in a node, the newly created node is connected to the node in which you dropped. If you drop outside any node, the new node is created at the root level.
  - m You can also drag and drop data taken from a Text widget, e.g. the scrolled-Text in the bottom pane. XmText widgets drag both **compoundText** and **String** targets. When you drop such data in the widget, the compoundText is handled as above. The String is checked for its format and, if it is not in tree format, it is not copied anywhere. Beware of pseudo tree formats.
5. drag and drop in overview mode - You can drag and drop from/to the tree widgets while they are in overview mode, using the same mechanisms as when the widgets are not in overview mode.

### 3.3 TreeEdit Example

The intent, in this demonstration, is to illustrate the widget's functionality as completely as possible. This program does not intend to be user friendly, but to give the user direct access to the widget's resources so that he can get a feel for the meaning of each resource.

The example is built with XFaceMaker. To launch the demo, change to the directory

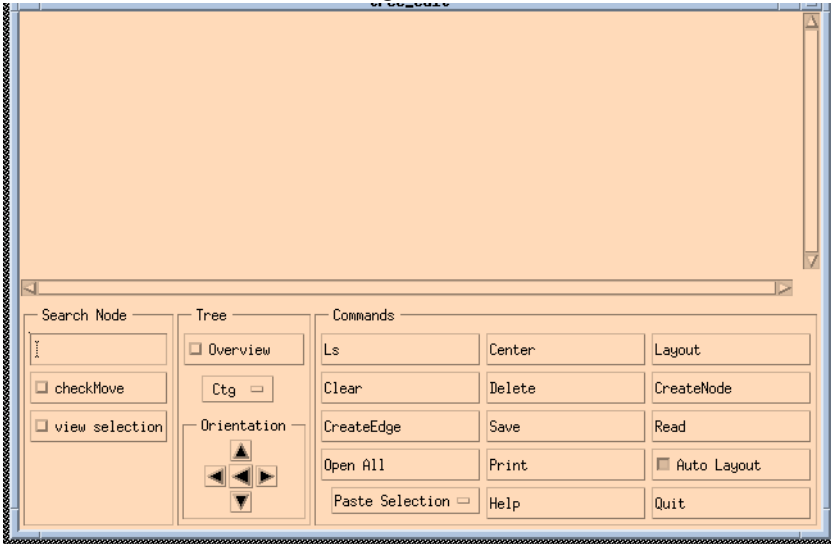
\$NSLHOME/demos/xnsltree/treeditt

and launch the executable



```
./tree_edit
```

Your screen should look something like this:



The application's main window comprises two parts; the upper half consists in a **scrolledTree** with which the user will interact using the buttons in the lower portion of the window. Among these buttons, there is a **Help** button which, as its name implies, pops a help window which describes most of the user interactions available. The **Quit** button which causes the demo to exit.

The source code for this demo is provided along with the executable. All resources are specified using XFaceMaker; callbacks are written using the FACE language, with, occasionally, a call to an application function. Resource settings for the main window and its callback scripts are in the file **Tree.fm**; application functions are in the file **tree.c**. The specialized editing boxes are in individual **.fm** files whose names correspond to their purpose. A Makefile helps you rebuild the demo.

Note, however, that the executable that is provided in the distribution is somewhat different from the one that you can rebuild: the color and font editing boxes in the original executable are implemented with the NSL color and font selection widgets, while they are implemented with the source **.fm** files when you rebuild the executable. Moreover, the original executable is built with an unprotected XnslTree library. If you rebuild the demo and link with a protected XnslTree library, the demo will request a license when you launch it.

### 3.3.1 User Interaction

1. **Loading a directory tree** - Click on the **Ls** button in the Commands frame in the lower part of the window. The current directory is loaded in the XnslTree widget. Note the different pixmaps, according to the type of file represented. Each node name is the name of the file represented.
2. **Expanding directories** - Double click Button 1 on a directory node (either the pixmap or the name). The directory sub-tree is expanded. You can expand additional levels, if there are any, either by double clicking in the directory nodes, or by clicking in the edge pixmap that looks like a '+' sign. Notice that it becomes a '-' sign when the node is expanded.
3. **Changing the layout algorithm** - Open the Option Menu in the Tree frame in the lower part of the window. Try the two other options: **Ortho** and **Isi**. Scroll the tree using the scroll bars to get a feel for the entire tree layout. Switch back to **Ctg** mode.
4. **Loading a large tree and using hand move** - Clear you current tree (**Clear** button in Commands frame) and Read a large tree description file: the **Read** button in the Commands frame opens a FileSelector - select **deep.tree**. Press button2 to initiate **hand move**: the cursor changes to the hand cursor and, as you move the mouse, the tree moves in the view window. Note that the scroll bar sliders move also: hand move is another way of scrolling the tree. By default, the widget implements hand move with a translation on button3 but, in this application, the translation table has been modified to implement hand move on button2, button3 being put to other uses (see below).
5. **Switching to Overview mode** - Toggle the **Overview** button. The entire tree becomes visible in the window, nodes now look like thin rectangles. A larger rectangle, representing the normal viewing window, is overlayed on the tree, at the current position of the viewing window. Drag the rectangle (with mouse button1) to the area that you will want to display in the normal window, and toggle the Overview button. Check that the nodes visible are indeed the ones you specified.
6. **Editing the XnslTree widget's resources** - Double click button1 in the widget, outside of any node or edge. An **Expert Box** is popped. It lets you specify a resource name and apply a value to it. If you don't know resource names (or don't like typing) open a resource popup menu by clicking button3 in the text-Field labelled **Resource**. Make your selection and close the menu, by clicking button3 on the desired resource name. Once you have a resource name, you can specify a value for it. Here again, you can open a value popup menu by clicking button3 in the value textfield. The value options change with the resource selected. However, the value option menus for the more complex resource

types have an ellipsis option which lets you open a special resource editing box. Try changing a color or a font and any other resources whose effect you want to see. Note that the **Auto Layout** toggle is set, so that you get immediate visual feedback for any change you make: each modification triggers a tree layout.

7. **Editing node attributes** - Click button3 on a node to open a popup menu that will let you open a node attributes editing box. The menu will include an item that says '*edit <nodename >*' if there is only one node selected, and '*edit selection*' if several nodes are selected. Click button3 in that menu item to edit a node. It's better to start off with just one node selected. You then get the current values for the node selected and you can modify them as desired. Beware: do not change the **selectable** attribute, or you will no longer be able to edit any of the attributes of that node. Edit all other attributes freely. Use the ellipsis buttons to open the custom editing boxes as needed. Once you have explored possibilities on a single node, you can try changing the attributes on several nodes simultaneously, by selecting several nodes, then opening the node attributes box on one of them. You can do multiple selection with rubberband selection, with the standard Motif Ctrl button1 mechanism, or use the **Search Node** text-Field: enter the leading characters of the node names you want to select and validate with Return.

If, instead of editing nodes on the file you loaded with **Read**, you try editing nodes on the tree built with the **Ls** button, the demo is able to check the type of file represented by the node and to adjust the menu contents accordingly. For example, you can travel the directory tree by clicking on the *chdir* menu item, if you opened the menu on a node that represents a directory. If you open the menu on an executable, you can launch it, if on a text file, you can edit it (if NSL's Wx editor is accessible via your PATH environment variable).

8. **Editing edge attributes** - Click button3 on an edge to open a popup menu that will let you open an edge attributes editing box. The menu includes an item that says '*edge\_att*'. Click button3 in the menu item to edit an edge's attributes. Start off with just one edge selected, to see the edge's current attribute values in the box. Use the ellipsis buttons to open the custom editing boxes as needed. Here again, avoid changing the selectable attribute, as there is no going back. Once you have explored possibilities on a single edge, you can try changing the attributes on several edges simultaneously, by selecting several edges, then opening the edge attributes box on one of them. You can do multiple selection with rubberband selection, with the standard Motif Ctrl button1 mechanism.
9. **Changing the tree orientation** - In the Orientation frame, try tree layout for various orientations by clicking on one of the arrow buttons in the Orientation frame. Note that the central arrow points in the direction of the current orientation.

10. ***Creating nodes and edges*** - Click on the ***CreateNode*** button to create a node. When you move the cursor over the tree, the cursor changes to a cursor that says *node*. Move the icon to the position where you want to put the node and click button. This opens the node attributes box. Set the attributes you want and click ***Create***. Click on the ***CreateEdge*** button to create an edge. The cursor changes to a cursor that says Src. Move the icon to the node that you want to specify as source for the edge and click button1. The cursor changes to Dst. Move it to the node that you want to specify as destination for the edge and click button1. This opens the edge attributes box. Set the attributes and click ***Create***.

---

## CHAPTER 4: XnslTree Functions

The functions that follow are in the XnslTree library.

They are presented in the following format:

### **NAME**

The C name of the function.

### **SYNOPSIS**

The calling syntax, the arguments, and the return type.

### **DESCRIPTION**

A brief description of the function.

### **EXAMPLE**

When available, provides a short example illustrating how the function can be used.

### **SCOPE**

States whether the function can be called from a FACE script only, from a FACE script and the application, or from the application only.

### NAME

**XnslAddTreeWidgetClass**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslAddTreeWidgetClass(fns)
FmFunctionsVector      *fns;
```

### DESCRIPTION

**XnslAddTreeWidgetClass** registers the XnslTree class in XFaceMaker, i.e. should be used to build a new XFaceMaker in which the XnslTree class is known. Returns 0 on success, and -1 on failure.

### EXAMPLE

```
#include <Xm/Xm.h>
#include <Fm.h>

int main (argc, argv)
int argc;
char    **argv;
{
    Widget w;
    FmFunctionsVector *fns;

    w = FmInitialize ("XnslTreefm", "XnslTreefm", 0, 0,
    &argc, argv);
    fns = FmGetFunctionsVector();
    XnslAddTreeWidgetClass(fns);
    FmCallEditor ();
}
```

### SCOPE

application

---

## NAME

### **XnslCreateScrolledTree**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

Widget XnslCreateScrolledTree
    (parent, name, arglist, argcount)
Widget      parent;
char        *name;
ArgList     arglist;
Cardinal    argcount;
```

## DESCRIPTION

**XnslCreateScrolledTree** creates an instance of a tree widget contained in a Scrolled Window. All the Scrolled Window components are automatically created.

The widget ID returned by the function is that of the Tree widget instance. To obtain the ID of the ScrolledWindow widget instance, use the *XtParent* function.

## SCOPE

application

### NAME

**XnslTreeCenterNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeCenterNode (w, node)
Widget                w;
XnslTreeNodeId       node;
```

### DESCRIPTION

**XnslTreeCenterNode** scrolls the tree so as to place node *node* in the center of the viewing window of tree *w*. Does nothing if the widget is not in an Xm-ScrolledWindow.

### SCOPE

Face, application



---

## NAME

**XnslTreeComplete**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
void XnslTreeComplete ()
```

## DESCRIPTION

**XnslTreeComplete** terminates the use of the Tree library. If the library contains license protection, calling its termination function releases its license token. Application programs may call these functions safely when using the unprotected versions of the libraries.

## SCOPE

Face, application

### NAME

**XnslTreeConnectString**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeConnectString
    (w, root, s, mask, attributes)
Widget                                w;
XnslTreeNodeId                       *root;
char                                  *s;
unsigned long                         mask;
XnslTreeEdgeAttributes               *attributes;
```

### DESCRIPTION

**XnslTreeConnectString** loads string *s* into the widget, connecting it as a sub-tree of node *root* if *root* is not NULL. *mask* and *attributes* specify the parameters of the edge created between *root* and the contents of *s*. If *root* is NULL, the function loads the contents of *s* at the root level. The format of the string is the one described in “File format” on page 40. The function returns 0 in case of success, -1 if it fails.

### SCOPE

Face, application

---

## NAME

**XnslTreeCopySelAndSubTrees**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
String XnslTreeCopySelAndSubTrees(w)  
Widget                                w;
```

## DESCRIPTION

**XnslTreeCopySelAndSubTrees** returns the String which contains the selection and the associated sub-trees.

## SCOPE

Face, application

### NAME

**XnslTreeCreateEdge**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
XnslTreeEdgeId  
    XnslTreeCreateEdge (w, from, to, mask, attributes)  
Widget          w;  
XnslTreeNodeId  from;  
XnslTreeNodeId  to;  
unsigned long    mask;  
XnslTreeEdgeAttributes *attributes;
```

### DESCRIPTION

**XnslTreeCreateEdge** creates an edge between node *from* and node *to*. *mask* specifies which parameters are valid in the *attributes* structure which is passed. Returns the ID of the edge created. If the resource `XnslNtreeCheckValidEdge` is `True`, the function checks that the edge meets the creation criteria. If the edge does not meet the criteria, it is not created and, the function returns `NULL`.

### SCOPE

Face, application

---

## NAME

**XnslTreeCreateNode**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

XnslTreeNodeId
    XnslTreeCreateNode(w, mask, attributes)
Widget          w;
unsigned long    mask;
XnslTreeNodeAttributes *attributes;
```

## DESCRIPTION

**XnslTreeCreateNode** creates a node. *mask* specifies which parameters are valid in the *attributes* structure which is passed. Returns the ID of the node created.

## SCOPE

Face, application

### NAME

**XnslTreeCutSelAndSubTrees**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
String XnslTreeCutSelAndSubTrees(w)  
Widget                                w;
```

### DESCRIPTION

**XnslTreeCutSelAndSubTrees** destroys the selection and returns the String which contains the selection and the associated sub-trees.

### SCOPE

Face, application

---

## NAME

**XnslTreeDeleteAllEdges**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteAllEdges(w)
Widget                                w;
```

## DESCRIPTION

**XnslTreeDeleteAllEdges** deletes all edges in tree *w*.

## SCOPE

Face, application

### NAME

**XnslTreeDeleteAllNodes**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteAllNodes(w)
Widget                                w;
```

### DESCRIPTION

**XnslTreeDeleteAllNodes** deletes all nodes and, as a consequence, all edges in tree *w*.

### SCOPE

Face, application



---

## NAME

**XnslTreeDeleteEdge**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteEdge (w, edge)
Widget                    w;
XnslTreeEdgeId           edge;
```

## DESCRIPTION

**XnslTreeDeleteEdge** deletes the edge whose ID is *edge* in tree *w*.

## SCOPE

Face, application

### NAME

**XnslTreeDeleteNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteNode (w, node)
Widget                    w;
XnslTreeNodeId           node;
```

### DESCRIPTION

**XnslTreeDeleteNode** deletes the node whose ID is *node* in tree *w*. Any edges associated to the node are deleted.

### SCOPE

Face, application

---

## NAME

### **XnslTreeDeleteNodeList**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteNodeList(w, nodes, count)
Widget                                w;
XnslTreeNodeId                        *nodes;
int                                    count;
```

## DESCRIPTION

**XnslTreeDeleteNodeList** deletes *count* nodes in tree *w*. Node ID's are passed in the nodes list *nodes*. Any edges associated to the nodes in the list are deleted.

## SCOPE

Face, application

### NAME

**XnslTreeDeleteSelection**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteSelection (w)
Widget      w;
```

### DESCRIPTION

**XnslTreeDeleteSelection** deletes all nodes and edges currently selected in tree *w*. Any un-selected edges associated to selected nodes are deleted as well.

### SCOPE

Face, application

---

## NAME

**XnslTreeDeleteSubtree**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeleteSubtree (w,node)
Widget          w;
XnslTreeNodeId  node;
```

## DESCRIPTION

**XnslTreeDeleteSubtree** deletes node *node* and its entire sub-hierarchy in tree *w*. Any edges associated to node *node* are also deleted.

## SCOPE

Face, application

### NAME

**XnslTreeDeselectAllEdges**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeselectAllEdges(w)
Widget                                w;
```

### DESCRIPTION

**XnslTreeDeselectAllEdges** deselects any selected edges in tree *w*.

### SCOPE

Face, application

---

## NAME

**XnslTreeDeselectAllNodes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeDeselectAllNodes(w)
Widget                                w;
```

## DESCRIPTION

**XnslTreeDeselectAllNodes** deselects any selected nodes in tree *w*.

## SCOPE

Face, application

### NAME

**XnslTreeEditNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeEditNode (w,node)
Widget                w;
XnslTreeNodeId       node;
```

### DESCRIPTION

**XnslTreeEditNode** overlays a text input widget on the label of node *node* in tree *w* to allow interactive editing of the node's name. The widget has only one text widget whose type depends on the **treeNodeEditMode** resource setting. If the resource is set to *Xns/NONE*, the function does nothing. If this function is called for several nodes in succession, the node specified in the last of the function calls will get the widget.

### SCOPE

Face, application



---

## NAME

**XnslTreeExpandNode**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeExpandNode (w, node)
Widget                w;
XnslTreeNodeId        node;
```

## DESCRIPTION

**XnslTreeExpandNode** expands node *node* in tree *w*, i.e. expands the sub-tree under node *node*. If the sub-tree has not been shrunk, the function does nothing.

## SCOPE

Face, application

### NAME

**XnslTreeExpandSelection**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeExpandSelection (w)
Widget                        w;
```

### DESCRIPTION

**XnslTreeExpandSelection** expands all selected nodes in tree *w*, i.e. expands the sub-tree under each node in the selection. Any sub-trees that have not been shrunk are left unchanged.

### SCOPE

Face, application

---

## NAME

**XnslTreeGetEdgeAttributes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void
XnslTreeGetEdgeAttributes(w, edge, mask, attributes)
Widget                               w;
XnslTreeEdgeId                       edge;
unsigned long                         mask;
XnslTreeEdgeAttributes               *attributes;
```

## DESCRIPTION

**XnslTreeGetEdgeAttributes** fetches the attributes of edge *edge* in tree *w*. *mask* specifies which fields of the edge attributes structure should be updated. If any attributes of type String were specified in *mask*, they should be freed when they are no longer needed.

## SCOPE

Face, application

### NAME

**XnslTreeGetEdgeEnds**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeGetEdgeEnds(w, edge, from, to)
Widget          w;
XnslTreeEdgeId  edge;
XnslTreeNodeId *from;
XnslTreeNodeId *to;
```

### DESCRIPTION

**XnslTreeGetEdgeEnds** retrieves the source and destination node ID's of edge *edge* in tree *w*. The source node is returned in *from* and the destination node in *to*.

### SCOPE

Face, application

---

## NAME

**XnslTreeGetEdgeFromTo**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
XnslTreeEdgeId XnslTreeGetEdgeFromTo(w, from, to)  
Widget                                     w;  
XnslTreeNodeId                             from;  
XnslTreeNodeId                             to;
```

## DESCRIPTION

**XnslTreeGetEdgeFromTo** returns the ID of the edge defined between node *from* and node *to* in tree *w*, if the edge exists. If there is no such edge, returns NULL.

## SCOPE

Face, application

### NAME

**XnslTreeGetEdges**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetEdges (w, edges, count)
Widget                w;
XnslTreeEdgeId        *edges[];
int                   *count;
```

### DESCRIPTION

**XnslTreeGetEdges** copies the ID's of all the edges in tree *w* into the array *edges* and returns the number of edges. It also copies the number of edges in *count*. If the number of edges is not zero, the application should free the *edges* array when it no longer needs it.

### SCOPE

Face, application

---

## NAME

### **XnslTreeGetEdgesFrom**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetEdgesFrom(w, node, from, count)
Widget                                w;
XnslTreeNodeId                       node;
XnslTreeEdgeId                       *from[];
int                                   *count;
```

## DESCRIPTION

**XnslTreeGetEdgesFrom** copies the ID's of all the edges connected from node *node* in tree *w* into the array *from* and returns the number of edges. It also copies the number of edges in *count*. If the number of edges is not zero, the application should free the *from* array when it no longer needs it.

## SCOPE

Face, application

### NAME

**XnslTreeGetEdgesTo**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetEdgesTo(w, node, to, count)
Widget                                w;
XnslTreeNodeId                       node;
XnslTreeEdgeId                       *to[];
int                                   *count;
```

### DESCRIPTION

**XnslTreeGetEdgesTo** copies the ID's of all the edges connected to node *node* in tree *w* into the array *to* and returns the number of edges. It also copies the number of edges in *count*. If the number of edges is not zero, the application should free the *to* array when it no longer needs it.

### SCOPE

Face, application



---

## NAME

**XnslTreeGetLeaves**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetLeaves(w, leaves, count)
Widget      w;
XnslTreeNodeId *leaves[];
int          *count;
```

## DESCRIPTION

**XnslTreeGetLeaves** copies the ID's of all the nodes in tree *w* which have no successors into the array *leaves*. The function returns the number of such nodes. It also copies the number of leaves in *count*. If the number of leaves is not zero, the application should free the *leaves* array when it no longer needs it.

## SCOPE

Face, application

## NAME

**XnslTreeGetNodeAttributes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void
XnslTreeGetNodeAttributes(w, node, mask, attributes)
Widget                                w;
XnslTreeNodeId                       node;
unsigned long                         mask;
XnslTreeNodeAttributes               *attributes;
```

## DESCRIPTION

**XnslTreeGetNodeAttributes** fetches the attributes of node *node* in tree *w*. *mask* specifies which fields of the node attributes structure should be updated. If any attributes of type String were specified in *mask*, they should be freed when they are no longer needed.

## SCOPE

Face, application

## EXAMPLE in a FACE script

```
XnslTreeCallbackStruct cbs;
XnslTreeNodeAttributes att;
cbs = $;
mask = 0;
mask = XnslTreeSetMaskField(mask,
                             XnslTreeNodeXMask);
att = new_struct(XnslTreeNodeAttributes);
nodeid = cbs -> node;
if (nodeid) {
    XnslTreeGetNodeAttributes (self, nodeid,
                               mask, att);

    att -> x = att -> x + 10;
    XnslTreeSetNodeAttributes (self, nodeid,
                               mask, att);
}
```

---

## NAME

### **XnslTreeGetNodes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetNodes(w, nodes, count)
Widget          w;
XnslTreeNodeId *nodes[];
int             *count;
```

## DESCRIPTION

**XnslTreeGetNodes** copies the ID's of all the nodes in tree *w* into the array *nodes* and returns the number of nodes. It also copies the number of nodes in *count*. If the number of nodes is not zero, the application should free the *nodes* array when it no longer needs it.

## SCOPE

Face, application

### NAME

**XnslTreeGetPixmap**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
Pixmap XnslTreeGetPixmap(w, name)
Widget                                w;
String                                name;
```

### DESCRIPTION

**XnslTreeGetPixmap** returns the pixmap whose name is *name*. *w* is the tree widget ID. *name* can be the name of an X11 standard bitmap or of an XPM format pixmap. The Motif function `XmInstallImage` is called to register the pixmap in the Motif cache.

### SCOPE

Face, application

---

## NAME

### **XnslTreeGetPixmapMask**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
Pixmap XnslTreeGetPixmapMask(w, name)
```

```
Widget                                w;
```

```
String                                name;
```

## DESCRIPTION

**XnslTreeGetPixmapMask** returns the bitmap whose name is *name*. *w* is the tree widget ID. *name* is the name of an X11 standard bitmap. The bitmap returned has a depth of 1.

## SCOPE

Face, application

### NAME

#### **XnslTreeGetPredecessors**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
int XnslTreeGetPredecessors(w, node, pred, count)  
Widget                                w;  
XnslTreeNodeId                       node;  
XnslTreeNodeId                       *pred[];  
int                                    *count;
```

### DESCRIPTION

**XnslTreeGetPredecessors** copies the ID's of all nodes in tree *w* which precede node *node* into the array *pred* and returns the number of nodes copied. It also copies the number of nodes in *count*. If the number of nodes is not zero, the application should free the *pred* array when it no longer needs it.

### SCOPE

Face, application

---

## NAME

### **XnslTreeGetRoots**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetRoots(w, roots, count)
Widget          w;
XnslTreeNodeId *roots[];
int             *count;
```

## DESCRIPTION

**XnslTreeGetRoots** copies the ID's of all root nodes in tree *w* into the array *roots* and returns the number of root nodes. A root node is a node that has no predecessor. The function also copies the number of root nodes in *count*. If the number of root nodes is not zero, the application should free the *roots* array when it no longer needs it.

## SCOPE

Face, application

### NAME

**XnslTreeGetSelectedEdges**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
int XnslTreeGetSelectedEdges(w, edges, count)  
Widget                                w;  
XnslTreeEdgeId                       *edges[];  
int                                    *count;
```

### DESCRIPTION

**XnslTreeGetSelectedEdges** copies the ID's of all selected edges in tree *w* into the array *edges* and returns the number of edges. The function also copies the number of selected edges in *count*. If the number of selected edges is not zero, the application should free the *edges* array when it no longer needs it.

### SCOPE

Face, application



---

## NAME

### **XnslTreeGetSelectedNodes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetSelectedNodes(w, nodes, count)
Widget                                w;
XnslTreeNodeId                       *nodes[];
int                                   *count;
```

## DESCRIPTION

**XnslTreeGetSelectedNodes** copies the ID's of all selected nodes in tree *w* into the array *nodes* and returns the number of nodes. The function also copies the number of selected nodes in *count*. If the number of selected nodes is not zero, the application should free the *nodes* array when it no longer needs it.

## SCOPE

Face, application

### NAME

**XnslTreeGetSuccessors**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeGetSuccessors(w, node, succ, count)
Widget                                w;
XnslTreeNodeId                       node;
XnslTreeNodeId                       *succ[];
int                                   *count;
```

### DESCRIPTION

**XnslTreeGetSuccessors** copies the ID's of all nodes in tree *w* which are successors of node *node* into the array *succ* and returns the number of nodes copied. The function also copies the number of successors in *count*. If the number of successors is not zero, the application should free the *succ* array when it no longer needs it.

### SCOPE

Face, application

---

## NAME

**XnslTreeHaveACycle**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
Boolean XnslTreeHaveACycle(w)  
Widget                                w;
```

## DESCRIPTION

**XnslTreeHaveACycle** checks tree *w* for cycles. Returns True if the graph has at least one cycle, False otherwise.

## SCOPE

Face, application

### NAME

**XnslTreeHighLightNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeHighLightNode(wid,node)
Widget          wid;
XnslTreeNodeId  node;
```

### DESCRIPTION

**XnslTreeHighLightNode** draws a rectangle surrounding node *node*. This function is used to depict a drag and drop effect when the pointer is inside a node in drag mode. It is called automatically when the **XnslTreeAutoHighlight** resource is set to True. Applications should call this function explicitly to render drag & drop effect when **XnslTreeAutoHighlight** is set to False.

### SCOPE

Face, application

---

## NAME

### **XnslTreeInNode**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

XnslTreeNodeId
    XnslTreeInNode(w, x, y, in_label, in_pixmap)
Widget          w;
int             x;
int             y;
int             *in_label;
int             *in_pixmap;
```

## DESCRIPTION

**XnslTreeInNode** returns the ID of the node under the pixel whose coordinates are *x* and *y*, or NULL if there is no node at this location of the viewing window of tree *w*. Additionally, if *x, y* is in the node label, the function returns 1 in *in\_label*, otherwise 0. If *x, y* is in the node pixmap, the function returns 1 in *in\_pixmap*, otherwise 0. If NULL has been passed in the *in\_label* and *in\_pixmap* arguments, the function does not check where *x* and *y* lie within the node.

## SCOPE

Face, application

### NAME

**XnslTreeIsTreeString**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
Boolean XnslTreeIsTreeString(s)  
String      s;
```

### DESCRIPTION

**XnslTreeIsTreeString** returns True if string *s* is a tree description string, i.e. conforms to tree file format; it returns false otherwise.

### SCOPE

Face, application

---

## NAME

**XnslTreeLayout**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
void XnslTreeLayout(w)  
Widget                w;
```

## DESCRIPTION

**XnslTreeLayout** triggers the layout calculations for tree *w* according to the current value of the **XnslNtreeLayoutAlgorithm** resource.

## SCOPE

Face, application

### NAME

**XnslTreeMatchedNames**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeMatchedNames(w, pattern, nodes, count)
Widget                                w;
char                                  *pattern;
XnslTreeNodeId                       *nodes[];
int                                   *count;
```

### DESCRIPTION

**XnslTreeMatchedNames** copies into the *nodes* array the node ID's of any nodes in tree *w* whose name matches the regular expression passed in *pattern*. The number of matching nodes is returned. It is also copied in *count*. If the number of matching nodes is not zero, the application should free the *nodes* array when it no longer needs it.

### SCOPE

Face, application

### SEE ALSO

XnslTreeMatchedNamesCi



---

## NAME

**XnslTreeMatchedNamesCi**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
int XnslTreeMatchedNamesCi(w, pattern, nodes, count)
Widget                                w;
char                                  *pattern;
XnslTreeNodeId                       *nodes[];
int                                    *count;
```

## DESCRIPTION

**XnslTreeMatchedNamesCi** copies into the *nodes* array the node ID's of any nodes in tree *w* whose name matches, the comparison being case insensitive, the regular expression passed in *pattern*. The number of matching nodes is returned. It is also copied in *count*. If the number of matching nodes is not zero, the application should free the *nodes* array when it no longer needs it.

## SCOPE

Face, application

## SEE ALSO

XnslTreeMatchedNames

### NAME

**XnslTreeNameToNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
XnslTreeNodeId XnslTreeNameToNode(wid, name)
Widget          wid;
char            *name;
```

### DESCRIPTION

**XnslTreeNameToNode** returns the first node whose name is *name* in tree widget *wid*.

### SCOPE

Face, application

### SEE ALSO

XnslTreeNameToNodeCi

---

## NAME

**XnslTreeNameToNodeCi**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
XnslTreeNodeId XnslTreeNameToNodeCi(wid, name)
Widget          wid;
char            *name;
```

## DESCRIPTION

**XnslTreeNameToNodeCi** returns the first node whose name is *name* in tree widget *wid*. The search is case insensitive.

## SCOPE

Face, application

## SEE ALSO

XnslTreeNameToNode

### NAME

**XnslTreeNodeIsVisible**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeNodeIsVisible(wtree, node)
Widget          wtree;
XnslTreeNodeId  node;
```

### DESCRIPTION

**XnslTreeNodeIsVisible** returns *zero* if node *node* in the XnslTree widget *wtree* is not mapped or any of its predecessors is not mapped. It returns 1 if, on the contrary, the node is mapped and all its predecessors are mapped, i.e the node may be displayed.

### SCOPE

Face, application

---

## NAME

**XnslTreeNodeWidget**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

Widget XnslTreeNodeWidget(node)
XnslTreeNodeId      node;
```

## DESCRIPTION

**XnslTreeNodeWidget** returns the Tree widget containing node *node*.

## SCOPE

Face, application

### NAME

**XnslTreeOnEdge**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
XnslTreeEdgeId  
XnslTreeOnEdge(w, x, y, in_label, in_pixmap)  
Widget          w;  
int             x;  
int             y;  
int             *in_label;  
int             *in_pixmap;
```

### DESCRIPTION

**XnslTreeOnEdge** returns the ID of the edge under the pixel whose coordinates are *x* and *y*, or NULL if there is no edge at this location in the viewing window of tree *w*. Additionally, if *x*, *y* is in the edge label, the function returns 1 in *in\_label*, otherwise 0. If *x*, *y* is in the edge pixmap, the function returns 1 in *in\_pixmap*, otherwise 0. If NULL has been passed in the *in\_label* and *in\_pixmap* arguments, the function does not check where *x* and *y* lie within the edge.

### SCOPE

Face, application

---

## NAME

### **XnslTreeReadFile**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeReadFile(w, filename)
Widget                                w;
char                                  *filename;
```

## DESCRIPTION

**XnslTreeReadFile** loads the tree stored in file *filename* into widget *w*. The format of the tree in *filename* is the one described in “File format” on page 40. The function returns 0 if it detects no errors, and -1 if it cannot load the file contents. The function adds the contents of *filename* to the existing contents of the widget.

## SCOPE

Face, application

### NAME

**XnslTreeReadString**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
int XnslTreeReadString(w, s)  
Widget                                w;  
char                                  *s;
```

### DESCRIPTION

**XnslTreeReadString** loads the tree contained in string *s* into widget *w*. The function returns 0 if it succeeds, -1 if it fails. The format of the string is the one described in “File format” on page 40. The contents of string *s* is added to the previous contents of widget *w*.

### SCOPE

Face, application



---

## NAME

**XnslTreeRedisplay**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeRedisplay(w)
Widget          w;
```

## DESCRIPTION

**XnslTreeRedisplay** redraws any regions of tree *w* that are not up-to-date.

## SCOPE

Face, application

### NAME

**XnslTreeRedisplayAll**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeRedisplayAll(w)
Widget                w;
```

### DESCRIPTION

**XnslTreeRedisplayAll** redraws all visible portions of tree *w*.

### SCOPE

Face, application

---

## NAME

### **XnslTreeReorderPredecessors**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeReorderPredecessors(w, node, nodes, count)
Widget                                w;
XnslTreeNodeId                        node;
XnslTreeNodeId                        *nodes;
int                                    count;
```

## DESCRIPTION

**XnslTreeReorderPredecessors** reorders the predecessors of node *node* in widget *w*. The predecessors of node *node* should be passed in *nodes*, in the order in which they are to be laidout. *count* is the number of nodes passed in *nodes*; it must be equal to the total number of predecessors of node. The function returns 0 if it detects no errors, and -1 if *count* is not equal to the number of predecessors of *node*, or if one of the nodes passed in *nodes* is not a predecessor of *node*.

## SCOPE

Face, application

### NAME

**XnslTreeReorderSuccessors**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
int XnslTreeReorderSuccessors(w, node, nodes, count)  
Widget                                w;  
XnslTreeNodeId                       node;  
XnslTreeNodeId                       *nodes;  
int                                    count;
```

### DESCRIPTION

**XnslTreeReorderSuccessors** reorders the successors of node *node* in widget *w*. The successors of node *node* should be passed in *nodes*, in the order in which they are to be laidout. *count* is the number of nodes passed in *nodes*; it must be equal to the total number of successors of node. The function returns 0 if it detects no errors, and -1 if *count* is not equal to the number of successors of *node*, or if one of the nodes passes in *nodes* is not a successor of *node*.

### SCOPE

Face, application

---

## NAME

**XnslTreeSavePsWidget**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeSavePsWidget(w, f, a)
Widget          w;
char            *f;
XnslTreePageAttributes *a;
```

## DESCRIPTION

**XnslTreeSavePsWidget** saves widget *w* in file *f* in Postscript format, according to the formatting specified in *a*. Make sure the page attributes structure, *a*, is fully initialized. The function returns 0 if successful, and -1 if an error occurs and the PostScript file could not be created.

## SCOPE

Face, application

### NAME

**XnslTreeSelectEdge**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSelectEdge(w, edge)
Widget          w;
XnslTreeEdgeId  edge;
```

### DESCRIPTION

**XnslTreeSelectEdge** adds edge *edge* to the edge selection of tree *w*.

### SCOPE

Face, application

---

## NAME

### **XnslTreeSelectionToString**

## SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
char *XnslTreeSelectionToString(w)  
Widget                                w;
```

## DESCRIPTION

**XnslTreeSelectionToString** copies the selection of widget *w* into the string returned. Returns 0 if the selection is empty. The format of the string is the one described in “File format” on page 40. The string returned is allocated by the function and should be freed when no longer needed. A selected edge is copied only if both its *from* and its *to* nodes are selected.

## SCOPE

Face, application

### NAME

**XnslTreeSelectNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSelectNode(w, node)
Widget                w;
XnslTreeNodeId        node;
```

### DESCRIPTION

**XnslTreeSelectNode** adds node *node* to the node selection of tree *w*.

### SCOPE

Face, application



---

## NAME

**XnslTreeSelectSubTree**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSelectSubTree(w, root
Widget                                w;
XnslTreeNodeId                       root;
```

## DESCRIPTION

**XnslTreeSelectSubTree** selects sub-tree starting at node *root* of tree *w*.

## SCOPE

Face, application

### NAME

**XnslTreeSetEdgeAttributes**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void
    XnslTreeSetEdgeAttributes(w, edge, mask, attributes)
Widget                                w;
XnslTreeEdgeId                       edge;
unsigned long                         mask;
XnslTreeEdgeAttributes                *attributes;
```

### DESCRIPTION

**XnslTreeSetEdgeAttributes** applies the attributes passed in *attributes* corresponding to the fields specified by *mask* to the edge whose ID is *edge*. *w* is the widget ID of the tree..

### SCOPE

Face, application

---

## NAME

**XnslTreeSetEdgeListAttributes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSetEdgeListAttributes
    (w, edges, count, mask, attributes)
Widget                                w;
XnslTreeEdgeId                       *edges;
int                                   count;
unsigned long                         mask;
XnslTreeEdgeAttributes               *attributes;
```

## DESCRIPTION

**XnslTreeSetEdgeListAttributes** applies the attributes passed in *attributes* corresponding to the fields specified by *mask* to the *count* edges whose ID's are passed in *edges*. *w* is the widget ID of the tree.

## SCOPE

Face, application

### NAME

**XnslTreeSetMaskField**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

unsigned long  XnslTreeSetMaskField(mask, bit)
unsigned long                                     mask;
unsigned                                             bit;
```

### DESCRIPTION

**XnslTreeSetMaskField** sets bit *bit* in mask *mask*. Use this function to build mask variables in FACE scripts.

### EXAMPLE

```
mask = 0;
mask = XnslTreeSetMaskField
      (mask, XnslTreeNodeNameMask);
```

### SCOPE

Face, application

---

## NAME

### **XnslTreeSetNodeAttributes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void
XnslTreeSetNodeAttributes(w, node, mask, attributes)
Widget                                w;
XnslTreeNodeId                       node;
unsigned long                         mask;
XnslTreeNodeAttributes               *attributes;
```

## DESCRIPTION

**XnslTreeSetNodeAttributes** applies the attributes passed in *attributes* corresponding to the fields specified by *mask* to the node whose ID is *node*. *w* is the widget ID of the tree.

## SCOPE

Face, application

## EXAMPLE in a FACE script

```
XnslTreeCallbackStruct cbs;
XnslTreeNodeAttributes att;
cbs = $;
mask = 0;
mask = XnslTreeSetMaskField(mask,
                             XnslTreeNodeXMask);
att = new_struct(XnslTreeNodeAttributes);
nodeid = cbs -> node;
if (nodeid) {
    XnslTreeGetNodeAttributes (self, nodeid,
                               mask, att);
    att -> x = att -> x + 10;
    XnslTreeSetNodeAttributes (self, nodeid,
                               mask, att);
}
```

### NAME

**XnslTreeSetNodeListAttributes**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSetNodeListAttributes(w, nodes,
                                   count, mask, attributes)
Widget                                w;
XnslTreeNodeId                       *nodes;
int                                  count;
unsigned long                         mask;
XnslTreeNodeAttributes               *attributes;
```

### DESCRIPTION

**XnslTreeSetNodeListAttributes** applies the attributes passed in *attributes* corresponding to the fields specified by *mask* to the *count* nodes whose ID's are passed in *nodes*. *w* is the widget ID of the tree.

### SCOPE

Face, application

---

## NAME

**XnslTreeSetSubTreeEdgeAttributes**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSetSubTreeEdgeAttributes(w, node,
                                     mask, attributes)
Widget                                w;
XnslTreeNodeId                       node;
unsigned long                         mask;
XnslTreeEdgeAttributes                *attributes;
```

## DESCRIPTION

**XnslTreeSetSubTreeEdgeAttributes** applies the attributes passed in *attributes* corresponding to the fields specified by *mask* to the edges of the subtree, starting at node *node*. If *node* is NULL, the attributes are applied to the entire tree. *w* is the widget ID of the tree

## SCOPE

Face, application

### NAME

**XnslTreeSetSubTreeNodeAttributes**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSetSubTreeNodeAttributes(w, node,
                                     mask, attributes)
Widget w;
XnslTreeNodeId node;
unsigned long mask;
XnslTreeNodeAttributes *attributes;
```

### DESCRIPTION

**XnslTreeSetSubTreeNodeAttributes** applies the attributes passed in *attributes* corresponding to the fields specified by *mask* to the nodes of the subtree, starting at node *node*. If *node* is NULL, the node attributes are applied to the entire tree. *w* is the widget ID of the tree.

### SCOPE

Face, application



---

## NAME

### **XnslTreeSetViewWindowPosition**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeSetViewWindowPosition(w,x,y)
Widget                                w;
int                                   x;
int                                   y;
```

## DESCRIPTION

**XnslTreeSetViewWindowPosition**, in overview mode, places the upper left corner of the view window rectangle at position  $x,y$  relative to the upper left corner of the complete tree. In standard mode, sets the scroll bar values of widget  $w$  to  $x$  and  $y$ .

## SCOPE

Face, application

### NAME

**XnslTreeShrinkNode**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
void XnslTreeShrinkNode(w, node)
Widget                                w;
XnslTreeNodeId                        node;
```

### DESCRIPTION

**XnslTreeShrinkNode** shrinks node *node*, in tree *w*. Shrinking a node consists in folding all of a node's successors onto the node.

### SCOPE

Face, application

---

## NAME

**XnslTreeShrinkSelection**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeShrinkSelection(w)
Widget                                w;
```

## DESCRIPTION

**XnslTreeShrinkSelection** shrinks all the nodes in the selection of tree *w*. Shrinking a node consists in folding all of a node's successors onto the node.

## SCOPE

Face, application

### NAME

**XnslTreeToString**

### SYNOPSIS

```
#include <Xnsl/Tree.h>

char *XnslTreeToString(w)
Widget                                w;
```

### DESCRIPTION

**XnslTreeToString** copies the tree contained in widget *w* into the string returned. Returns 0 if the widget is empty. The format of the string is the one described in “File format” on page 40. The string returned is allocated by the function and should be freed when no longer needed.

### SCOPE

Face, application

---

## NAME

**XnslTreeUnHighLightNode**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

void XnslTreeUnHighLightNode (wid,node)
Widget          wid;
XnslTreeNodeId  node;
```

## DESCRIPTION

**XnslTreeUnHighLightNode** deletes the rectangle surrounding the node *node*, the rectangle having been previously drawn by a call to the function **XnslTreeHighLightNode**.

## SCOPE

Face, application

### NAME

**XnslTreeUnsetMaskField**

### SYNOPSIS

```
#include <Xnsl/Tree.h>
```

```
unsigned long  XnslTreeUnsetMaskField(mask, bit)
unsigned long                                     mask;
unsigned                                              bit;
```

### DESCRIPTION

**XnslTreeUnsetMaskField** unsets bit *bit* in mask *mask*. Use this function to build mask variables in FACE scripts.

### SCOPE

Face, application

---

## NAME

**XnslTreeWriteFile**

## SYNOPSIS

```
#include <Xnsl/Tree.h>

int XnslTreeWriteFile(w, filename)
Widget      w;
char        *filename;
```

## DESCRIPTION

**XnslTreeWriteFile** saves the tree contained in widget *w* into file *filename*. The format of the tree in *filename* is the one described in “File format” on page 40. The function returns 0 if it detects no errors, and -1 if it fails.

## SCOPE

Face, application





## Index

Symbols

%B substitution character 11

### A

actions 32

application

    build 2

    compiled interface 4

    interpreted interface 3

attributes

    edge 36

    node 34

### B

build

    application 2

### C

callback

    treeActivateCallback 10, 16

    treeChangeNameCallback  
        10, 17

    treeDeleteCallback 10, 18

    treeDragCallback 10, 18

    treeDropCallback 10, 18

    treeInputCallback 10, 20

    treeLoadUserDataCallback  
        10, 21

    treeMoveCallback 10, 22

    treeMoveVerifyCallback 10,  
        23

    treeSaveUserDataCallback  
        10, 26

    treeSelectionCallback 10, 27

class

    xnslTreeWidgetClass 7

constants

    for attributes 43

    for callback structures 43

    for resources 41

### D

decrypt

    code 1

development library 1

drag

    selection 9

    String 9

drag and drop 9

drop

    Compound Text 9

    String 9

### E

edge

    characteristics 8

environment variable 10

    NSLHOME 11

    XBMLANGPATH 11

    XNSL\_TREE\_RGB\_DB 10

### F

file

    rgb.txt 10

    treergb.txt 11

    XnsITree format 40

Fm library 4

Fm\_c library 5

format

    XnsITree file 40

### L

layout

- CTG 7
- ISI 7
- ORTHOGONAL 7
- library
  - decryption code 1
  - development 1
  - Fm 4
  - Fm\_c 5
  - run-time 1
  - Xpm 10
- license server 1
- N
- network license server 1
- node
  - characteristics 8
- NSLHOME 11
- NSLlicense 1
- O
- orientation
  - XnslBOTTOM\_TOP 25
  - XnslLEFT\_RIGHT 25
  - XnslRIGHT\_LEFT 25
  - XnslTOP\_BOTTOM 25
- R
- reason
  - XnslCR\_LOAD\_EDGE\_USER\_DATA 21
  - XnslCR\_LOAD\_NODE\_USER\_DATA 21
  - XnslCR\_SAVE\_EDGE\_USER\_DATA 26
  - XnslCR\_SAVE\_NODE\_USER\_DATA 26
  - XnslCR\_TREE\_ACTIVAT
    - E 16
  - XnslCR\_TREE\_CHANGE\_NAME\_VERIFY 17
  - XnslCR\_TREE\_DELETE 18
  - XnslCR\_TREE\_DRAG 18
  - XnslCR\_TREE\_DRAG\_VERIFY 18
  - XnslCR\_TREE\_DROP\_CTER 19
  - XnslCR\_TREE\_DROP\_ENAVE 19
  - XnslCR\_TREE\_DROP\_MOTION 19
  - XnslCR\_TREE\_DROP\_STRING 19
  - XnslCR\_TREE\_DROP\_VERIFY 19
  - XnslCR\_TREE\_INPUT 20
  - XnslCR\_TREE\_MOVE 22
  - XnslCR\_TREE\_MOVE\_VERIFY 23
  - XnslCR\_TREE\_NAME\_CHANGED 18
  - XnslCR\_TREE\_SELECTIOIN 27
- rgb.txt file 10
- run-time library 1
- S
- structure
  - constants for 43
  - XnslTreeCallbackStruct 31
  - XnslTreePageAttributes 39

- 
- T
  - translations 31
  - treergb.txt file 11
  - X
  - XBMLANGPATH 11
    - %B 11
  - XFaceMaker
    - extended 3
    - extending 2
  - xfm 2
  - Xnsl\_REDISPLAY\_APPLICATION 25
  - Xnsl\_REDISPLAY\_ON\_TOPLLEVEL 25
  - Xnsl\_REDISPLAY\_SYSTEMATIC 25
  - Xnsl\_RESIZE\_ON\_TOPLEVEL 25
  - Xnsl\_RESIZE\_SYSTEMATIC 25
  - XNSL\_TREE\_RGB\_DB 10
  - XnslAddTreeWidgetClass** 56
  - XnslAS\_NEEDED 27
  - XnslBOTTOM\_TOP 25
  - XnslCR\_LOAD\_EDGE\_USER\_DATA 21
  - XnslCR\_LOAD\_NODE\_USER\_DATA 21
  - XnslCR\_SAVE\_EDGE\_USER\_DATA 26
  - XnslCR\_SAVE\_NODE\_USER\_DATA 26
  - XnslCR\_TREE\_ACTIVATE 16
  - XnslCR\_TREE\_CHANGE\_NAME\_VERIFY 17
  - XnslCR\_TREE\_DELETE 18
  - XnslCR\_TREE\_DRAG 18
  - XnslCR\_TREE\_DRAG\_VERIFY 18
  - XnslCR\_TREE\_DROP\_CTEXT 19
  - XnslCR\_TREE\_DROP\_ENTER 19
  - XnslCR\_TREE\_DROP\_LEAVE 19
  - XnslCR\_TREE\_DROP\_MOTION 19
  - XnslCR\_TREE\_DROP\_STRING 19
  - XnslCR\_TREE\_DROP\_VERIFY 19
  - XnslCR\_TREE\_INPUT 20
  - XnslCR\_TREE\_MOVE 22
  - XnslCR\_TREE\_MOVE\_VERIFY 23
  - XnslCR\_TREE\_NAME\_CHANGED 18
  - XnslCR\_TREE\_SELECTION 27
  - XnslCreateScrolledTree** 57
  - XnslCTG 7, 20
  - XnslCtrl 3
  - XnslFm 3
  - XnslFROM\_ARROW 17
  - XnslFROM\_TO\_ARROW 17
  - XnslINT\_USER\_DATA 21, 26
  - XnslISI 7, 20
  - XnslLEFT\_RIGHT 25

XnsIMULTIPLE\_LINE 24  
XnsINO\_ARROW 17  
XnsINONE 24  
XnsINScrollbarDisplayPolicy 27  
XnsINtreeActivateCallback 16  
XnsINtreeArrowHeight 16  
XnsINtreeArrows 16  
XnsINtreeArrowWidth 16  
XnsINtreeAutoHighlight 17  
XnsINtreeAutoLayout 17  
XnsINtreeAutomaticMode 17  
XnsINtreeBorder 17  
XnsINtreeChangeNameCall-  
back 17  
XnsINtreeCheckValidEdge 18  
XnsINtreeDeleteCallback 18  
XnsINtreeDoHandMove 18  
XnsINtreeDoRubberband 18  
XnsINtreeDragCallback 18  
XnsINtreeDropCallback 18  
XnsINtreeEdgeFont 19  
XnsINtreeEdgeRatioAutoCom-  
pute 19  
XnsINtreeFile 20  
XnsINtreeFillArrow 20  
XnsINtreeHeight 20  
XnsINtreeInputCallback 20  
XnsINtreeLayoutAlgorithm 20  
XnsINtreeLoadUserDataCall-  
back 21  
XnsINtreeMargin 22  
XnsINtreeMinusPixmap 22  
XnsINtreeMoveCallback 22  
XnsINtreeMoveVerifyCallback  
23  
XnsINtreeNodeAltPixmap 23  
XnsINtreeNodeAltPixmap-  
Mask 23  
XnsINtreeNodeDisplayPolicy 23  
XnsINtreeNodeEditMode 24  
XnsINtreeNodeFont 24  
XnsINtreeNodeHeight 24  
XnsINtreeNodePixmap 24  
XnsINtreeNodePixmapMask 24  
XnsINtreeNodeWidth 24  
XnsINtreeNormalCursor 24  
XnsINtreeOrientation 24  
XnsINtreeOverview 25  
XnsINtreeOverviewMargin 25  
XnsINtreePixmapLabelSep 25  
XnsINtreePlusPixmapName 25  
XnsINtreeRedisplayMethod 25  
XnsINtreeResizeMethod 25  
XnsINtreeSaveUserDataCall-  
back 26  
XnsINtreeSelectionCallback 27  
XnsINtreeSpace 27  
XnsINtreeType 27  
XnsIORTHOGONAL 7, 20  
XnsIRIGHT\_LEFT 25  
XnsISHOW\_ALL 23  
XnsISHOW\_LABELS 23  
XnsISHOW\_NONE 23  
XnsISHOW\_PIXMAPS 23  
XnsISINGLE\_LINE 24  
XnsIStand 3  
XnsISTATIC 27  
XnsISTRING\_USER\_DATA

- 21, 26
- XnslTO\_ARROW 17
- XnslTOP\_BOTTOM 25
- XnslTree 7
  - actions 32
  - extension 2
  - in XFaceMaker 2
  - translations 31
- XnslTreeCallbackStruct 31
- XnslTreeCenterNode 58
- XnslTreeComplete 1, 2
- XnslTreeComplete 59
- XnslTreeConnectString 60
- XnslTreeCopySelAndSubTrees 61
- XnslTreeCreateEdge 62
- XnslTreeCreateNode 63
- XnslTreeCutSelAndSubTrees 64
- XnslTreeDeleteAllEdges 65
- XnslTreeDeleteAllNodes 66
- XnslTreeDeleteEdge 67
- XnslTreeDeleteNode 68
- XnslTreeDeleteNodeList 69
- XnslTreeDeleteSelection 70
- XnslTreeDeleteSubtree 71
- XnslTreeDeselectAllEdges 72
- XnslTreeDeselectAllNodes 73
- XnslTreeEdgeAttributes 36
- XnslTreeEditNode 74
- XnslTreeExpandNode 75
- XnslTreeExpandSelection 76
- XnslTreeGetEdgeAttributes 77
- XnslTreeGetEdgeEnds 78
- XnslTreeGetEdgeFromTo 79
- XnslTreeGetEdges 80
- XnslTreeGetEdgesFrom 81
- XnslTreeGetEdgesTo 82
- XnslTreeGetLeaves 83
- XnslTreeGetNodeAttributes 84
- XnslTreeGetNodes 85
- XnslTreeGetPixmap 86
- XnslTreeGetPixmapMask 87
- XnslTreeGetPredecessors 88
- XnslTreeGetRoots 89
- XnslTreeGetSelectedEdges 90
- XnslTreeGetSelectedNodes 91
- XnslTreeGetSuccessors 92
- XnslTreeHaveACycle 93
- XnslTreeHighLightNode 94
- XnslTreeInNode 95
- XnslTreeIsTreeString 96
- XnslTreeLayout 97
- XnslTreeMatchedNames 98
- XnslTreeMatchedNamesCi 99
- XnslTreeNameToNode 100
- XnslTreeNameToNodeCi 101
- XnslTreeNodeAttributes 34
- XnslTreeNodeIsVisible 102
- XnslTreeNodeWidget 103
- XnslTreeOnEdge 104
- XnslTreePageAttributes 39
- XnslTreeReadFile 105
- XnslTreeReadString 106
- XnslTreeRedisplay 107
- XnslTreeRedisplayAll 108
- XnslTreeReorderPredecessors 109
- XnslTreeReorderSuccessors 110
- XnslTreeSavePsWidget 111
- XnslTreeSelectEdge 112

**XnslTreeSelectionToString** 113  
**XnslTreeSelectNode** 114  
**XnslTreeSelectSubTree** 115  
**XnslTreeSetEdgeAttributes** 116  
**XnslTreeSetEdgeListAttributes** 117  
**XnslTreeSetMaskField** 118  
**XnslTreeSetNodeAttributes** 119  
**XnslTreeSetNodeListAttributes** 120  
**XnslTreeSetSubTreeEdgeAttributes** 121  
**XnslTreeSetSubTreeNodeAttributes** 122  
**XnslTreeSetViewWindowPosition** 123  
**XnslTreeShrinkNode** 124  
**XnslTreeShrinkSelection** 125  
**XnslTreeToString** 126  
**XnslTreeUnHighLightNode** 127  
**XnslTreeUnsetMaskField** 128  
**xnslTreeWidgetClass** 7  
**XnslTreeWriteFile** 129  
Xpm library 10